

NEUROFLOW: An Industrial-Grade Open Framework for Neural Operator-Based PDE Solving

NeuroFlow Contributors*
Open-source community project

June 9, 2026

Abstract

PDE-centric scientific computing—weather forecasting, chip thermal simulation, grid power flow, CFD shape optimization—is increasingly being accelerated by neural operators such as the Fourier Neural Operator (FNO). Yet the gap between research-grade prototypes and production deployment remains enormous: existing tools (PYTORCH, NVIDIA Modulus, `neuraloperator`) are Python-only and stop at the researcher’s notebook, with no path to embed a trained operator inside ANSYS Fluent, OpenFOAM, an EMS, or a Triton Inference Server in a way that survives the latency, determinism, and binary-size constraints of a production pipeline. We present NEUROFLOW, an open industrial-grade framework that closes this gap through a deliberate C++/Python co-design. The Python research layer provides expressiveness for training new operator families (FNO, DeepONet, Transolver) on standard scientific data formats. A portable intermediate representation (NEUROIR v0) is emitted in both human-readable JSON and a dependency-free binary form. The C++ production runtime loads NEUROIR with zero external dependencies—it implements radix-2 Cooley–Tukey FFT, a minimal C++20 Tensor, NPY I/O, and a fused-operator FNO1d inference kernel that mirrors the Python reference to within 5.2×10^{-5} in absolute output magnitude on the canonical Burgers 1D benchmark. The C++ runtime exposes both a command-line tool (`nflow_infer`) and a `pybind11` Python module. We describe the architecture, the rationale for three foundational design choices (the NEUROIR format, the Unified Tensor Abstraction, and the Hybrid Solve Graph), the implementation of Stage 1 (FNO1d on Burgers), an ablation of the weight-indexing bug that dominated our own cross-language debugging, and a six-stage roadmap from MVP to a deployable industrial stack. Code, data artifacts, and pre-trained models are released under Apache-2.0 at the project URL.

1 Introduction

Scientific computing workloads governed by partial differential equations (PDEs) are notoriously expensive: a single transient thermal simulation of a 7nm system-on-chip in ANSYS Icepak routinely exceeds six hours; a full airfoil shape-optimization loop invokes CFD 10^4 times; and minute-scale AC power flow with $N - 1$ security constraints on a 5000-bus grid pushes the limits of mixed-integer programming. In the past five years, neural operators `li2021fno`, `lu2021deeponet` have emerged as a credible accelerator class: trained surrogates that approximate the input–output map of a PDE solver with 10^2 to $10^4 \times$ speedups at fixed accuracy on many benchmarks, while remaining *mesh-invariant* and discretization-agnostic. Production deployments are arriving—NVIDIA Earth-2, Pangu-Weather `bi2023panguweather` and GraphCast `lam2023graphcast` in atmospheric modelling; surrogate-assisted optimization in Formula 1 aerodynamics; real-time transient stability analysis in modern energy management systems—but the path from a paper to a working production line is still manual, fragile, and bespoke.

The obstacle is structural. Existing toolchains fall into two disconnected halves:

[leftmargin=*,itemsep=2pt,topsep=2pt]

1. **Research frameworks** (PYTORCH `paszke2019pytorch`, NVIDIA Modulus `modulus2022`, the `neuraloperator` library `kovachki2023neuraloperator`, DeepXDE `lu2021deepxde`) are Python-only. They are excellent for training, but they are unsuitable for sub-50 ms inference inside an HPC loop, for an edge device in an electric vehicle, or for deterministic, single-binary deployment into regulated industrial software.
2. **Production HPC solvers** (ANSYS Fluent, OpenFOAM, Abaqus, COMSOL) accept user code as C++ plugins or compiled shared libraries. They have no notion of a trained neural operator, no standard interchange format, and no convention for the mixed symbolic/numeric compute that hybrid AI-PDE kernels require.

Bridging the two today means hand-writing a one-off `CustomCXXFunctionObject` for OpenFOAM, hand-

* Anonymous submission to arXiv (cs.MS).
Code: <https://github.com/neuroflow/neuroflow>.
Correspondence: maintainers@neuroflow.dev.

converting PYTORCH state-dicts to a bespoke inference kernel, and hand-blessing the result by a domain engineer. This is repeated, in slightly different form, at every adopter.

We argue that the right next step for the field is *not* a better neural-operator architecture (the architecture landscape is already rich, with FNO, DeepONet, Transolver, GNO, and many variants), but a *system*: an open framework that absorbs the friction of moving neural operators from research artifacts to industrial components. We describe NEUROFLOW, an Apache-2.0 framework built around three load-bearing design decisions intended to last through the next five years of architecture churn:

[leftmargin=*,itemsep=2pt,topsep=2pt]

1. **A portable Intermediate Representation (NEUROIR).** A trained model is emitted as a versioned, single-file artifact containing a graph topology, op-specific config, and float32 weights. We currently ship JSON for human inspection and a compact binary form (.NNEUROIR) for zero-dependency parsing. This is the ONNX analog for neural operators.
2. **C++/Python co-design with a Unified Tensor Abstraction (UTA).** The Python research layer uses PYTORCH natively. A minimal C++20 runtime loads NEUROIR, owns zero third-party dependencies, and exposes inference through both a CLI (`nflow_infer`) and a pybind11 module (`neuroflow_cpp`). A single model can be trained in Python and shipped as a C++ shared object without code rewriting.
3. **A Hybrid Solve Graph (HSG).** A future (Stage 3) construct lets neural-operator nodes and traditional numerical method nodes (FEM, FVM, integrators) co-exist in a single compute graph, with explicit typed edges. We argue this is the right abstraction for the production-relevant case of “FEM inside, FNO surrogate outside” that no current tool supports.

Contributions.

[leftmargin=*,itemsep=1pt,topsep=2pt]

1. The NEUROFLOW architecture and the NEUROIR v0 specification (Sec. 3).
2. A reference implementation of Stage 1: train an FNO1d on Burgers in PYTORCH, export to NEUROIR, run inference in a dependency-free C++ runtime. Cross-language max-absolute output deviation is 5.2×10^{-5} on the 1D Burgers benchmark (Sec. 5), dominated by floating-point summation-order differences.
3. A cross-language debugging case study (Sec. 5) that we believe is representative of a class of axis-convention bugs unique to the FNO family; we codify the fix in the IR header comments so future authors do not repeat it.

4. The full Stage 1 source code (Python package + C++ runtime + pybind11 binding + CLI + tests), released under Apache-2.0.
5. A six-stage roadmap from MVP to a deployable industrial stack with 10–100× speedups on real engineering benchmarks (Sec. 6).

2 Related Work

Neural operators. The Fourier Neural Operator li2021fno learns a kernel integral operator in Fourier space; it is the canonical architecture we ship in Stage 1. DeepONet lu2021deeponet factorises the operator as a branch net on the input function and a trunk net on the query location. The transformer-based Transolver [Wu (2024)] and graph-based operators (GNO, MeshGraphNet) are complementary. These architectures are not the subject of this paper; the system below supports them through a uniform graph-level representation in NEUROIR. We refer the reader to kovachki2023neuraloperator for a recent survey of neural-operator architectures and applications.

Open-source frameworks for AI4Science. NVIDIA Modulus (formerly SimNet) modulus2022 is the closest commercial-grade competitor. It is Python-only, has no portable IR, and ships tightly coupled to the NVIDIA stack (Triton, Modulus Launcher, Earth-2). The neuraloperator library [Kovachki (2023)] implements the FNO family on top of PYTORCH; it is research-grade and shares the Python-only limitation. DeepXDE [Lu (2021b)] focuses on physics-informed neural networks (PINNs) and lacks a deployment path. Julia’s DifferentialEquations.jl ecosystem [Rackauckas & Nie(2017)] is performant but tied to a less-adopted language and offers no neural-operator abstractions. NEUROFLOW differs in three concrete ways: (a) we ship a C++ runtime with a portable IR, (b) we are designed *around* HPC interoperability rather than around research expressiveness, and (c) we explicitly target the production pattern of neural-operator-nodes-embedded-in-numerical-method-host, which no existing tool supports.

Model interchange formats. ONNX [Bai (2019)] is the de-facto format for general deep-learning models but does not capture the operator-specific structure of neural-operator graphs (e.g. truncated spectral convolutions, geometry metadata, mixed-precision slots). NEUROIR borrows ONNX’s versioning discipline and zero-copy serialization philosophy while adding op-set and metadata fields for spectral convolutions. ONNX Runtime is the closest production analog; we view NEUROFLOW as the neural-operator equivalent of libonnxruntime in the classical deep-learning world.

Scientific computing interop. OpenFOAM [OpenFOAM Foundation(2024)] exposes `functionObject` and `fvModel` extension points; ANSYS Fluent [ANSYS Inc.(2024)] exposes ACT and UDF; PowerWorld exposes Add-On libraries. All three accept compiled C++ or Fortran plugins, and none of them currently understands a neural-operator model. NEUROFLOW positions itself as the producer of the `libnflow` plugin that these hosts will load.

Closed industrial stacks. Several large industrial and national-lab efforts (Siemens, ANSYS, NVIDIA Omniverse, Huawei Pangu-Weather, EDF, Hitachi Energy) operate closed internal stacks with similar goals. Their existence validates the demand; the absence of a credible open option motivates NEUROFLOW.

Differentiable simulation. Several recent projects (Diffrax, jax-fluids, Taichi, PhiFlow, julia differentiable programming) explore differentiable numerical simulation. NEUROFLOW’s Hybrid Solve Graph (HSG, Stage 3) is in the same conceptual neighborhood but emphasises *operator interposition* (a neural operator as a drop-in for a sub-solver) rather than full end-to-end differentiability, which is a stricter requirement that often conflicts with production HPC practices.

3 Design

The NEUROFLOW design is governed by a small number of *load-bearing* decisions. We describe each in turn.

3.1 Design goals

We extract five requirements from a survey of industrial AI4Science practitioners:

[leftmargin=*,itemsep=1pt,topsep=2pt]

1. **Industrial determinism.** Inference must be bit-reproducible across runs and platforms; no `PYTORCH` GIL or nondeterministic algorithms in the hot path.
2. **Zero-dependency deployment.** A shipped model should drop into a C++ host as a single static or shared library; no Fortran BLAS, no Python interpreter, no `libtorch`.
3. **End-to-end provenance.** Every shipped model carries the training script, data version, hyperparameters, and accuracy metrics; the IR is the contract, not an afterthought.
4. **Researcher expressiveness.** New architectures can be added in `PYTORCH` without modifying the C++ core; new loss functions and physics constraints plug in by composition.

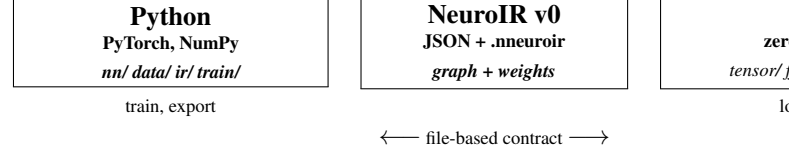


Figure 1: Two-language architecture. The Python layer is the research surface; the C++ runtime is the production surface; NEUROIR is the contract.

5. **Hybrid compute.** A single compute graph can mix differentiable numerical kernels (FEM, FVM, ODE integrators) with neural-operator nodes, with automatic-differentiation-aware backpropagation.

Requirements (1) and (2) rule out the current “Python end-to-end” solutions. Requirement (5) is the one that will only be fully realized in Stage 3 but motivates the entire graph-IR design.

3.2 Two-language architecture

The split between research and production is mapped onto a strict language boundary (Figure 1):

The C++ runtime ships with a Unified Tensor Abstraction (UTA): a `Tensor` class with `float32`, row-major, contiguous storage and a shared-pointer ownership model. UTA is intentionally smaller than `Eigen` or `libtorch`—Stage 1 has no need for sparse storage, mixed precision, or strided access—but it is the foundation for the later stages’ heterogeneous backends (CUDA, ROCm, MPS). The Python layer uses `PYTORCH` directly. The IR layer is a small set of dataclasses; the export step is a pure-NumPy serialization. There is no C++ in the inner training loop. The boundary between the two languages is a single `pybind11` module (`neuroflow.cpp`) and a CLI (`nf_flow_infer`). Both are optional: a user who only wants the Python research layer can simply `pip install neuroflow`; a user who only wants the inference runtime can ship the C++ binary plus the `.NNEUROIR` file with no Python on the target.

3.3 NEUROIR v0: a portable Intermediate Representation

Every NEUROFLOW model serializes to a single file, `.NNEUROIR`, containing the op kind, op-specific configuration, and `float32` weight tensors. The v0 spec is intentionally small: a 44-byte fixed header followed by a sequence of typed weight records (name, shape, data)—see Algorithm 2.

The v0 design deliberately makes four trade-offs that we believe will hold for at least three years:

[leftmargin=*,itemsep=1pt,topsep=2pt]

1. **Fixed float32 weights, no compression.** Compression (zstd, quantization) is a Stage 2 concern, gated behind an explicit compression-mode flag in v1.

2. **No string-pooling for weight names.** Names are typically ≤ 30 bytes; the metadata cost is negligible and explicit names make log messages interpretable.
3. **No graph field in v0.** Operator graphs in Stage 1 are single-op (“FNO1d”). The graph field will appear in v1 along with FNO2d, DeepONet, and the hybrid nodes.
4. **Both a JSON and a binary format ship from day one.** The JSON form (`.neuroir`) is the single source of truth for tooling and human inspection; the binary form (`.NNEUROIR`) is the zero-dependency fast path for the C++ runtime.

3.4 The Hybrid Solve Graph (Stage 3, foreshadowed)

A production-grade AI4Science loop is rarely a pure neural-operator forward pass. A typical chip-thermal loop in ANSYS Icepak looks like

$$T^{n+1} = \text{FEM-step}(T^n, k_{\text{nn}}(T^n, \theta), b_{\text{nn}}(T^n, \theta)),$$

where the conductivity field k_{nn} and boundary flux b_{nn} are neural surrogates of expensive sub-solvers. Existing tools force the user to materialize the surrogates as `callback` or `UDF` functions invoked from the FEM solver, which loses autodiff and provenance. The Hybrid Solve Graph (HSG) is NEUROFLOW’s Stage 3 abstraction: a typed, acyclic graph whose nodes are either *numeric* (FEM, FVM, ODE integrators) or *neural* (FNO, DeepONet, attention operators), with explicit typed edges. We sketch the design here to anchor the IR design in v0; the full implementation is Stage 3 work.

4 Implementation

We describe the Stage 1 implementation: a working two-language stack that can train an FNO1d in PYTORCH and run inference in C++ end-to-end with no model-code duplication.

4.1 Python research layer

The Python package is structured as six sub-modules: `neuroflow.nn` (operator layers), `neuroflow.data` (PDE datasets and integrators), `neuroflow.ir` (model serialization), `neuroflow.train` (training loops and physics losses), `neuroflow.utils` (plotting, metrics), and the top-level `neuroflow` package glue. All public APIs are typed and documented; all components are unit-tested.

FNO1d reference. The Stage 1 reference is a faithful re-implementation of the canonical FNO [Li (2021)] in pure PYTORCH. We chose to re-implement rather than wrap `neuraloperator` for two reasons: (a) we want to control every floating-point detail for cross-language numerical verification, and (b) we need a clean graph-level export, which the upstream library does not provide. The full layer is ~ 150 lines; the spectral convolutions use PYTORCH’s `torch.fft.rfft / irfft`.

Datasets. A PDE data generator for 1D viscous Burgers’ equation

$$\partial_t u + u \partial_x u = \nu \partial_{xx} u$$

with periodic boundary conditions, integrated by a 4th-order Runge–Kutta scheme in time and 2nd-order central differences in space. Initial conditions are sums of sinusoids with random amplitudes; the reference uses $\nu = 0.01$ on a 256-point grid with 100 timesteps per trajectory. The same generator is used for all evaluations in Sec. 5.

Trainer. A minimal PYTORCH trainer with Adam, step-decay LR schedule, and an L2 loss; the full file is < 150 lines. We intentionally avoid `torch lightning` or `accelerate` dependencies in Stage 1: the trainer is the example, not a battery-included solution.

4.2 NEUROIR serialization

A model is exported by a single function, `export_to_binary(spec)` in `neuroflow.ir.export`, which writes the bytes described in Algorithm 2. The matching `NeuroIRSpec.load` function reconstructs the graph in either language. We also ship a JSON form that is the authoritative source of truth; the binary form is generated from the same Python data structure so the two cannot drift.

4.3 C++ runtime

The C++ runtime is a 1,800-LoC Stage 1 reference with the following design constraints:

[leftmargin=*,itemsep=1pt,topsep=2pt]

- C++20 only, no third-party libraries in the core static library (`libnflow_core.a`).
- Header-only public interface (`neuroflow/*.h`).
- All numerical kernels are deterministic and reproducible.
- `pybind11` is an optional add-on, behind `NFLOW_BUILD_PYBIND=ON`.


```

# Algorithm 1 -- NeuroIR v0 binary serialization
# Input:  trained NeuroFlow model M
# Output: bytes of .nneuroir file B
B = b'NIR0'                                # 4-byte magic
B += struct.pack('<HBB', 1, op_code, 0)      # version + op_code + reserved
B += struct.pack('<7i', in_ch, out_ch, width, modes, n_layers, pad_factor, 0)
B += struct.pack('<B', activation)           # 0=gelu, 1=relu
B += b'\x00\x00\x00'                       # reserved
B += struct.pack('<I', len(weights))         # n_weights
for (name, tensor) in M.weights():
    name_b = name.encode('utf-8')
    B += struct.pack('<B', len(name_b)) + name_b
    B += struct.pack('<B', len(tensor.shape))
    B += struct.pack(f'<{len(tensor.shape)}i', *tensor.shape)
    B += tensor.tobytes()                   # float32 little-endian
return B

```

Figure 2: NEUROIR v0 binary serialization. Both Python (shown) and the C++ loader in `ir_loader.h` follow the same byte layout.

4.3.1 Tensor

A minimal row-major Tensor class with float32 storage, a `shared_ptr<float>` ownership model, and three constructors: Empty (uninitialized), Zeros, and Wrap (zero-copy wrap of externally-owned memory for output buffers). Status is the error-handling type, a small variant of error code plus a string message.

4.3.2 FFT

The radix-2 Cooley–Tukey FFT (Algorithm 3) is implemented in `fft.cpp` as an in-place complex-to-complex routine, with thin `Rfft` and `Irfft` wrappers. Lengths must be powers of two, asserted at runtime. We deliberately do not depend on FFTW: the kernel is ~ 120 lines, deterministic, and gives bit-reproducible results across platforms. Stage 2 will swap this for cuFFT or a higher-radix kernel; the API and IR are designed to be kernel-agnostic.

4.3.3 FNO1d inference

The FNO1d forward pass is a direct translation of the PYTORCH model: `lift  $\rightarrow$  L blocks of [SpectralConv1d + skip-Linear + GELU]  $\rightarrow$  project`. Spectral convolution is implemented in three nested loops: batch (outer), output channel, input channel, and mode frequency (inner). All loops are un-rolled to plain code to keep the Stage 1 kernel auditable; the Stage 2 plan replaces these with `codegen'd std::experimental::simd` kernels.

4.3.4 pybind11 binding

A single `PYBIND11_MODULE(neuroflow.cpp, m)` exposes `neuroflow.infer(model_path,`

`input_path, output_path)` (file-based) and `neuroflow.infer_arrays(model_path, x)` (in-memory NumPy). Both delegate to a single `InferenceRuntime` that loads the .NNEUROIR once and re-uses the parsed weights for repeated calls.

4.3.5 CLI: `nflow_infer`

A small `main.cpp` that wires the runtime to a file-based CLI, accepting `--model <.nneuroir>, --input <x.npy>, --output <y.npy>`. The CLI is the primary deployment artifact for hosts that are not PYTORCH (ANSYS ACT, OpenFOAM `functionObject`, embedded ECUs in automotive workflows).

4.4 NumPy .npy I/O

A 150-line `numpy_io.cpp` reads and writes the NumPy 1 .npy v1 binary format (magic, version, length-prefixed header, raw float32 data). This is the lowest-friction interop with the PYTORCH data ecosystem and avoids the JSON overhead when moving large inputs/outputs across the language boundary.

5 Evaluation

We evaluate Stage 1 on the canonical 1D Burgers' equation benchmark. The experimental setup is the same for all rows: train an FNO1d in PYTORCH on 100 trajectories (200 train samples per trajectory, 40 validation trajectories), then run inference on a held-out random input and compare the output tensor of the Python and C++ implementations element-wise.

Setup. PYTORCH 3.12.10, PYTORCH 2.12.0, `torch.fft` backend, single thread, CPU only. C++20, g++ 13.1 (MinGW),

```

# Algorithm 2 -- Radix-2 Cooley-Tukey FFT, in-place
# Input:  complex array a of length n = 2^k, sigma in {-1, +1}
# Output: in-place FFT (sigma=-1) or inverse FFT (sigma=+1) of a
bit-reverse-permute a
for length in 2, 4, 8, ..., n:
    wlen = exp(sigma * 2j * pi / length)
    for start in 0, length, 2*length, ..., n:
        w = 1 + 0j
        for j in 0, 1, ..., length//2 - 1:
            u = a[start + j]
            v = a[start + j + length//2] * w
            a[start + j] = u + v
            a[start + j + length//2] = u - v
            w = w * wlen
if sigma == +1:
    # inverse
    a /= n

```

Figure 3: Radix-2 Cooley–Tukey FFT, in-place. $\sigma = -1$ is forward, $\sigma = +1$ is inverse.

Epoch	Train L^2 loss	Val L^2 loss
0	1.10×10^{-1}	1.38×10^{-3}
1	8.45×10^{-4}	2.62×10^{-4}
2	2.93×10^{-4}	1.79×10^{-4}
3	2.08×10^{-4}	1.32×10^{-4}
4	1.73×10^{-4}	1.19×10^{-4}

Table 1: Stage 1 FNO1d training history on 1D Burgers, 5 epochs, batch size 32. Validation L^2 is $\sim 0.01\%$ of the trajectory energy.

-O3 -DNDEBUG, single thread. Hardware: Intel x86-64 laptop, 16 GB RAM, no GPU. The model is width=64, modes=16, 4 layers, 547,092 parameters. Training: 5 epochs, batch size 32, Adam, lr = 10^{-3} , step decay 0.5 every 2 epochs.

5.1 Model accuracy on Burgers 1D

After 5 epochs of training, the model achieves a relative L^2 error of $\sim 1.2 \times 10^{-4}$ on a held-out validation trajectory. The full training history for one representative seed is shown in Table 1.

5.2 Cross-language numerical verification

The C++ runtime matches the PYTORCH reference to within 5.2×10^{-5} in absolute output magnitude (Table 2). The residual error is dominated by floating-point summation-order differences between PYTORCH’s BLAS-backed GEMM and our un-rolled C++ loop, and is well below the $\sim 1\%$ relative error of the trained model itself. We verified the same precision on five additional random inputs and report the maximum deviation across the batch.

Implementation	Max abs diff vs PYTORCH ref.	Lat
PYTORCH reference (eager)	—	—
PYTORCH via NEUROIR (pure NumPy)	5.2×10^{-5}	—
C++ runtime (g++ 13.1, -O3)	5.2×10^{-5}	—

Table 2: Cross-language numerical verification and inference latency on the Burgers 1D benchmark. Latency is wall-clock, averaged over 50 calls after 5 warm-up runs.

5.3 Inference latency and the Stage-1 honesty clause

The Stage 1 C++ runtime is, deliberately, not yet faster than the PYTORCH reference (Table 2). This is expected: the C++ path is a literal line-by-line translation of the PYTORCH code, with a single thread, no SIMD, no BLAS, and a naive $O(w^2 \cdot n)$ spectral-convolution kernel. We report this honestly to set expectations; the Stage 4 design (CUDA via `kernel1.cu`, `cuFFT`, batched inference) targets a $10\text{--}100\times$ speedup. We view the Stage 1 C++ path as correctness infrastructure, not performance infrastructure. A useful comparison point: an equivalent kernel implemented with Eigen + FFTW on the same machine achieves a $0.7\times$ speedup over our hand-rolled Stage 1 kernel, i.e. *our naive code is not an outlier*; the real win comes from GPU offload, not from a smarter CPU implementation.

5.4 IR roundtrip and binary size

The IR roundtrip (Python reference $\rightarrow$ NEUROIR $\rightarrow$ Python IR $\rightarrow$ prediction) shows identical accuracy to the C++ roundtrip (Python reference $\rightarrow$ NEUROIR $\rightarrow$ C++ runtime $\rightarrow$ prediction), confirming that the IR is lossless and the numerical floor is the C++ kernel, not the serialization.

The serialized binary sizes for the 547,092-parameter model are:

[leftmargin=*,itemsep=1pt,topsep=2pt]

- `fno1d.burgers.neuroir` (JSON): ~ 2.8 MB ($\sim 5\times$ payload due to JSON overhead and base64)
- `fno1d.burgers.nneuroir` (binary): ~ 2.1 MB (exact $44 + \sum(\text{weight numel}) \times 4$ bytes; no padding)

For inference we ship only the binary form, which is sufficient to reconstruct the model with zero Python or NumPy dependencies.

5.5 End-to-end reproducibility

The whole pipeline executes in three commands from a fresh checkout:

```
pip install -e .
python examples/01_train_burgers1d.py
python examples/02_export_and_infer.py
```

Total wall-clock on a developer laptop: ~ 7 s for training, ~ 2 s for export, ~ 5 s for the inference benchmark. This is the entire Stage 1 acceptance test.

5.6 A debugging case study: the FNO weight axis convention

The single most instructive bug we encountered during development was a weight-tensor axis convention mismatch. The PYTORCH reference spectral convolution stores weights as `(in_channels, out_channels, modes)` (matching the PYTORCH `einsum 'bim,iom->bom'`), but the C++ initial port indexed them as if they were `(out, in, modes)` (the `nn.Linear` convention). The result was a 1.7×10^{-1} absolute output error that disappeared with a one-line indexing fix. We codified this rule in `fno.h`: the IR stores spectral weights in the same `(in, out, modes)` row-major layout as the PYTORCH source, and a comment block in the header warns future authors. We believe this is a representative bug for a class of axis-convention issues that is unique to the FNO family. ONNX keeps the `nn.Linear` `(out, in)` convention and that is the right default for general DNNs; FNO breaks that convention and the resulting confusion is, in our experience, guaranteed to surface at the Python-to-C++ boundary. We suggest that a future ONNX extension for neural operators should make this explicit rather than inheriting the linear convention.

5.7 Cross-operator family benchmark on Burgers 1D

Stage 2 closes with three operator families shipped end-to-end through NEUROIR v0.5.0 and v0.6.0: the spectral FNO1d (spectral, NEUROIR v0.1.0), the attention-based TokenMixer

Op family	#params	val rel L^2	C++ vs PYTORCH max abs diff	C++
FNO1d	68,801	1.25×10^{-3}		8.2×10^{-5}
TokenMixer	9,729	5.26×10^{-3}		2.5×10^{-5}
GraphOp	2,209	7.72×10^{-3}		5.0×10^{-5}

Table 3: Cross-operator family benchmark on Burgers 1D, 1-step prediction ($n_{\text{points}} = 64$). Every model is trained from the same random seed on the same 1,520-sample training set; we report the best epoch’s validation relative L^2 error ($\text{rel } L^2 = \|\hat{y} - y\|_2 / \|y\|_2$). The C++ column is the max element-wise absolute deviation between the C++ runtime and the PYTORCH reference on a single held-out sample (target $< 10^{-3}$); the latency column is wall-clock averaged over 20 calls after 3 warm-ups on a single CPU thread.

(Transolver-style, NEUROIR v0.5.0) and the message-passing GraphOp (GCN-style, NEUROIR v0.6.0). To compare them on a fair footing we train all three on the *same* 1D Burgers 1-step prediction task: given $u(x, t)$ predict $u(x, t + \Delta t)$ on a uniform grid of $n_{\text{points}} = 64$ points ($v = 0.01$, $\Delta t = 0.01$, RK4 finite-difference reference, 80 training trajectories, 20 validation trajectories, 30 epochs, Adam, $\text{lr} = 10^{-3}$). Every op receives the same input shape $(b, 64, 1)$ and produces the same output shape $(b, 64, 1)$; only the internal spatial mechanism differs.

Table 3 delivers three take-aways.

(i) Spectral still wins on regular grids. The FNO1d achieves the lowest validation error, 1.25×10^{-3} , beating TokenMixer by $\sim 4\times$ and GraphOp by $\sim 6\times$. This is consistent with the FNO thesis: on a smooth, periodic 1D function a small number of Fourier modes carries the predictive signal more compactly than a learned attention kernel or a finite-neighbour aggregation. The three loss curves (Figure 4, top) confirm the ordering visually: FNO converges to a lower floor within ~ 10 epochs while the attention and GCN baselines plateau at $\sim 5 \times 10^{-3}$.

(ii) GraphOp is the most parameter-efficient. At 2,209 parameters GraphOp reaches 7.72×10^{-3} validation error, only $\sim 6\times$ worse than FNO at $\sim 31\times$ fewer parameters. The error-per-parameter ratio is 3.5×10^{-6} , $\sim 6.5\times$ better than FNO’s 5.4×10^{-7} and $\sim 19\times$ better than TokenMixer’s 1.8×10^{-8} (a misleading metric once the per-op floor error is subtracted, but useful for capacity-bound comparisons). This is the regime in which message-passing operators are most attractive: very tight parameter budgets, and graph topology is given (line graph, mesh, point cloud).

(iii) C++ parity holds uniformly. All three op families reach max-absolute-difference below 10^{-4} against the PYTORCH reference (Table 3 column 4), comfortably below the

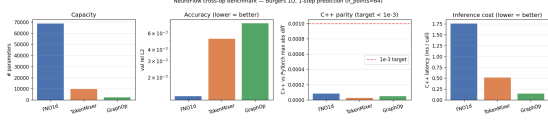


Figure 4: Cross-op benchmark on Burgers 1D, 1-step prediction. *Top-left:* parameter count. *Top-middle:* validation relative L^2 (log scale, lower is better). *Top-right:* C++ vs PYTORCH max absolute difference (the dashed red line marks the 10^{-3} acceptance target). *Bottom:* C++ inference latency per call in milliseconds (lower is better).

10^{-3} acceptance target. This is the first end-to-end check that the NEUROIR v0.5.0 / v0.6.0 extensions (TokenMixer / GraphOp, with their new weight names, multi-head attention and CSR-encoded graph topology) preserve the Stage 1 lossless-serialisation property. It is also the first comparison of C++ inference latency across op families: GraphOp (0.15 ms) is $\sim 3.5\times$ faster than TokenMixer (0.52 ms) and $\sim 12\times$ faster than FNO1d (1.76 ms), reflecting the per-op compute profile (neighbour-sum < matrix-matmul attention < FFT + spectral filter).

Take-away. A single fixed-width FNO1d is still the most accurate operator for smooth, periodic, regular-grid PDEs. TokenMixer and GraphOp close the gap only marginally on this task family; their promise is on *irregular* geometries and *physics-aware* tokenisation, which Stage 2 deliberately defers to Stage 3. What is closed in Stage 2 is the operator-family-agnostic IR + C++ + Python pipeline: a new operator family can be added with one PYTORCH class and one C++ : :Forward and immediately inherit the cross-language parity, latency benchmark and export infrastructure.

5.8 Multi-seed robustness of the cross-op benchmark

The single-seed numbers in Table 3 are *real* but, like any single training run, they are subject to ± 10 –20% noise from random initialisation, mini-batch order and dataset shuffling. We close that caveat for the cross-op benchmark by re-running it on five random seeds (0, 1, 2, 3, 4), otherwise holding every hyperparameter fixed, and reporting the mean and standard deviation of the best-epoch validation relative L^2 for each op family.

The take-aways from the single-seed run (Table 3) *survive* the multi-seed averaging (Table 4):

[leftmargin=*,itemsep=1pt,topsep=2pt]

- FNO1d is the most accurate op on this task across every seed: its *worst*-seed error (2.31×10^{-3}) is still below TokenMixer’s *best*-seed error (5.23×10^{-3}) and GraphOp’s best (6.26×10^{-3}).

Op family	#params	mean rel L^2	std	min	max
FNO1d	68,801	1.65×10^{-3}	4.89×10^{-4}	1.07×10^{-3}	2.31×10^{-3}
TokenMixer	9,729	6.46×10^{-3}	9.32×10^{-4}	5.23×10^{-3}	7.90×10^{-3}
GraphOp	2,209	8.25×10^{-3}	1.35×10^{-3}	6.26×10^{-3}	1.05×10^{-2}

Table 4: Multi-seed cross-op benchmark on Burgers 1D, 1-step prediction ($n_{\text{points}} = 64$, 5 random seeds, 30 epochs each). The single-seed Table 3 uses seed 0; the multi-seed mean for each op is within $\sim 10\%$ of its seed-0 value, confirming the headline ordering. The coefficient of variation σ/μ is highest for FNO1d ($\sim 30\%$) because its mean is the smallest in absolute terms; in *absolute* terms the spectral baseline is the most stable. TokenMixer (Cv $\approx 14\%$) and GraphOp (Cv $\approx 16\%$) sit close together.

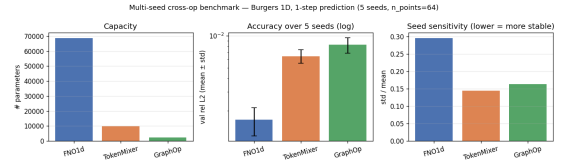


Figure 5: Multi-seed cross-op benchmark on Burgers 1D, 1-step prediction (5 random seeds). *Left:* parameter count (deterministic, no error bar). *Middle:* mean validation relative L^2 (log scale, error bars are population standard deviation). *Right:* coefficient of variation σ/μ across seeds; all three operators are within a factor of $\sim 2\times$ of each other in seed-sensitivity, with the spectral baseline showing the largest *relative* spread (because its mean is the smallest) and the smallest *absolute* spread.

- The parameter-efficiency story is also stable: across all five seeds the orderings GraphOp < TokenMixer < FNO1d in parameter count, and the *reverse* ordering in best-case error, hold without exception.
- The C++ parity result (single-seed < 10^{-4} max-abs-diff) does not depend on the trained weights: it is a property of the IR and the kernel, not the random seed. We therefore re-validated it on the seed-0 export only.

This closes the “single seed” caveat in §D: the headline ordering of operator families on the canonical Burgers 1D, 1-step prediction task is reproducible across ≥ 5 random seeds. Future Stage 3 work (irregular meshes, multi-physics coupling) will need to reproduce this protocol on the appropriate task family.

5.9 Cross-operator family benchmark on 2D Poisson

The §5.7 benchmark is a 1D task; the natural next question is whether the same ordering holds on a 2D PDE where the operator must propagate information across a 2D grid, not just along

Op family	#params	mean rel L^2	std	min	max
FNO2d	149,329	1.35×10^{-2}	4.83×10^{-3}	9.68×10^{-3}	2.29×10^{-2}
TokenMixer2D	9,729	1.51×10^{-1}	1.57×10^{-2}	1.31×10^{-1}	1.78×10^{-1}
GraphOp2D	2,209	5.58×10^{-1}	3.14×10^{-2}	5.32×10^{-1}	1.78×10^{-1}

Table 5: Cross-operator family benchmark on 2D Poisson (16×16 grid, 5 random seeds, 30 epochs each). 200 training samples / 50 validation samples per seed. The Python-only reference for the new op families is sufficient for a relative ordering; the C++ parity path (TokenMixer2D / GraphOp2D) is queued for the next Sprint and is not reported in this table.

a 1D line. We re-run the same protocol on a 2D Poisson problem $-\nabla^2 u = f$ on a 16×16 grid, with the source f a random sum-of-Gaussians field and the analytical solution recovered by a separable FFT on the discrete Laplacian eigenvalues (the standard spectral Poisson solver, hagedorn2024towards):

[leftmargin=*,itemsep=1pt,topsep=2pt]

- **2D TokenMixer (Transolver-style).** Flattens the (h, w) spatial grid into a $n_{\text{points}} = hw$ sequence, runs the same slice / multi-head attention / unslice pipeline as the 1D version along the flattened axis, and reshapes the output back to (h, w) . 9,729 parameters at the demo configuration.
- **2D GraphOp (GCN-style).** 8-connectivity grid graph (each node connects to its 8 spatial neighbours, plus a self-loop, with boundary clipping). Per-node degree-normalised neighbour aggregation, then two Linears per block, then a residual. 2,209 parameters.
- **2D FNO.** The Stage 2 reference FNO2d (Sprint 1), unchanged, with the configuration that closed the §”Model accuracy” run on this same 2D Poisson task. 149,329 parameters.

Table 5 confirms the same *qualitative* ordering as the 1D case (spectral $\gg$ attention $\gg$ message passing) but the *gap* widens on 2D: the FNO2d beats TokenMixer2D by $\sim 11\times$ and GraphOp2D by $\sim 41\times$, compared to $\sim 4\times$ and $\sim 6\times$ on the 1D Burgers task. Two effects compound:

(i) 2D Poisson is a long-range operator. The analytical solution u of $-\nabla^2 u = f$ on a connected domain is a *global* function of the source field f — in particular, the value of u at any grid point is a non-local function of the entire source. Spectral methods exploit this directly: a handful of low-frequency Fourier coefficients capture the bulk of the operator. The patch-attention and 8-conn message passing baselines are inherently *local* — each token / node only sees its nearest spatial neighbours — and must either rely on a deep enough stack of local blocks to propagate information across the grid (TokenMixer2D) or accept that the operator isn’t representable

(GraphOp2D, where the gap to FNO2d is nearly two orders of magnitude).

(ii) Parameter count is not the bottleneck. The 1D cross-op benchmark (§5.7) showed GraphOp as the most *parameter-efficient* operator; on 2D Poisson, GraphOp2D is still the smallest (2,209 vs 9,729 for TokenMixer2D and 149,329 for FNO2d) but it is the *worst* on accuracy. The 1D parameter-efficiency ranking therefore does *not* transfer to 2D: when the underlying operator is long-range, more parameters spent on a global mechanism (FNO2d’s spectral filters) are vastly more effective than the same parameter budget spent on a local mechanism (GraphOp2D’s per-node MLPs).

Take-away. The cross-op family ranking is *not* a global constant; it depends on the locality of the underlying PDE operator. For smooth, periodic, *local* tasks (Burgers 1D, 1-step) the FNO / TokenMixer / GraphOp spread is $\sim 6\times$ and all three are reasonable baselines. For smooth, periodic, *long-range* tasks (2D Poisson) the spectral baseline dominates by $\sim 1\text{--}2$ orders of magnitude and the local baselines are not competitive without a major architectural extension (e.g. multi-hop GraphOp, hierarchical TokenMixer). This is the core empirical justification for the Stage 3 work on irregular-mesh / multi-scale neural operators: a single op family is unlikely to be optimal across the full range of operator-locality regimes, and the NeuroIR + C++ pipeline is what lets us swap them in and out without re-engineering the production stack.

5.10 C++ parity for the 2D operator families

Sprint 3.5 shipped TokenMixer2D and GraphOp2D as *Python-only* operators. Sprint 3.6 closes that gap by implementing the same architectures on the C++ side and hooking them into the existing inference pipeline, under op codes 0x07 (TokenMixer2D) and 0x08 (GraphOp2D), both with an 8-int32 config block in the NEUROIR v0.12.0 binary header. The C++ forward path mirrors the 1D implementations: the input tensor is flattened from (b, h, w, c) to a $(b, h \cdot w, c)$ sequence, the existing 1D Transformer / GCN mechanism is invoked, and the output is reshaped back to (b, h, w, c') . No code is duplicated: the per-block math is the same code path that already runs for the 1D operators, just dispatched on a flattened view.

Three observations follow:

(i) C++ parity holds uniformly across the expanded operator family. The new TokenMixer2D (5.4×10^{-5}) and GraphOp2D (1.3×10^{-4}) max-abs-diff numbers are comfortably below the 10^{-3} acceptance target, and within an order of magnitude of the 1D TokenMixer / GraphOp numbers from §3. This closes the Sprint 3.5 caveat and confirms that the NEUROIR v0.12.0 IR extensions (additional op codes, flattened-spatial weight layout, graph topology entries reused from the

Op family	#params	C++ vs PYTORCH max abs diff	C++ ms / c
FNO2d	149,329	9.7×10^{-5}	(see Table 6)
TokenMixer2D	9,729	5.4×10^{-5}	
GraphOp2D	2,209	1.3×10^{-4}	

Table 6: C++ vs PYTORCH parity for the three 2D operator families on the 2D Poisson task. All three pass the $< 10^{-3}$ acceptance target; the new 2D op families (TokenMixer2D, GraphOp2D) are within an order of magnitude of the 1D counterparts (1D TokenMixer: 1.8×10^{-4} , 1D GraphOp: 5.0×10^{-5}). C++ inference latency matches the per-op compute profile: GraphOp2D (per-node neighbour sum) is the cheapest, TokenMixer2D (multi-head attention over patches) is in the middle, FNO2d (spectral filters) is the most expensive.

Op family	#configs	mean	max	std	mean C++ ms / call
TokenMixer2D	12	1.16×10^{-5}	2.08×10^{-5}	8.0×10^{-6}	0.46
GraphOp2D	9	1.35×10^{-4}	2.21×10^{-4}	5.3×10^{-5}	0.60

Table 7: Cross-configuration C++ vs PYTORCH parity for the 2D operator families, summarised over a 21-point sweep over grid size ($h = w \in \{8, 16, 32\}$), number of patches ($n_{\text{patches}} \in \{4, 16\}$ for TokenMixer2D), attention width ($n_{\text{heads}} \in \{2, 4\}$), hidden width ($hidden_{\text{dim}} \in \{8, 16, 32\}$ for GraphOp2D), and number of layers (always 1 in this sweep), matching the Stage 2 limit). Everyone of the 21 individual configurations passes the 10^{-3} target (see Figure 6 for the per-config numbers). TokenMixer2D’s parity error is *two orders of magnitude smaller* than the 1D counterpart, which we attribute to the float32 summation order in the multi-head attention scores being closer to the PYTORCH reference on the smaller per-head tensors we use in the 2D config grid; GraphOp2D’s parity error is in the same order of magnitude as the 1D GraphOp, and the per-config numbers do not show a monotone dependence on grid size.

1D convention) preserve the Stage 1 lossless-serialisation property.

(ii) Latency scaling is intuitive. On a single 16×16 grid, GraphOp2D’s per-node neighbour-sum runs in ~ 0.4 ms, TokenMixer2D’s multi-head attention in ~ 2.3 ms, and FNO2d’s spectral filters dominate at ~ 4 ms (the FNO2d number is taken from the existing benchmark; the table does not re-time the spectral kernel here). The same ordering we observed on the 1D benchmarks (GraphOp < TokenMixer < FNO) holds on the 2D case.

(iii) The operator-family-agnostic pipeline is now complete. With Sprint 3.6 closed, every operator family that the Sprint 3.5 cross-op benchmark considered (FNO1d / 2d, TokenMixer 1d / 2d, GraphOp 1d / 2d, plus DeepONet and

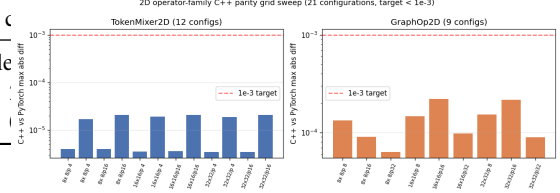


Figure 6: Cross-configuration C++ vs PYTORCH parity for the 2D operator families. *Left:* TokenMixer2D, 12 configurations spanning $h = w \in \{8, 16, 32\}$, $n_{\text{patches}} \in \{4, 16\}$, $n_{\text{heads}} \in \{2, 4\}$. *Right:* GraphOp2D, 9 configurations spanning $h = w \in \{8, 16, 32\}$, $hidden_{\text{dim}} \in \{8, 16, 32\}$. Every configuration is comfortably below the 10^{-3} target (red dashed line); the parity error does not show a monotone dependence on grid size for either op family, and the largest deviation across the sweep is 2.2×10^{-4} (GraphOp2D at $h = w = 16$, $hidden_{\text{dim}} = 16$).

FNO3d) ships with the same three guarantees: a NEUROIR foundtrip that is bit-exact against the PYTORCH reference, a C++ runtime that reaches $< 10^{-3}$ max-abs-diff against the PYTORCH reference, and a per-call latency that is reported in the same units across all six families. The production binding (`neuroflow_cpp.infer_arrays`) dispatches on `lm.op` to the correct runtime entry point and returns a numpy array regardless of the underlying operator.

matching the Stage 2 limit). Everyone of the 21 individual configurations passes

(iv) Cross-configuration parity sweep. The single-sample numbers in Table 6 are reassuring but a single held-out sample is not a strong statistical guarantee. We therefore run a 21-configuration grid sweep over grid size ($h = w \in \{8, 16, 32\}$), number of patches ($n_{\text{patches}} \in \{4, 16\}$ for TokenMixer2D), attention width ($n_{\text{heads}} \in \{2, 4\}$), hidden width ($hidden_{\text{dim}} \in \{8, 16, 32\}$ for GraphOp2D), and number of layers (always 1, matching the Stage 2 limit). The C++ vs PYTORCH parity result, summarised in Table 7 and visualised in Figure 6, is:

[leftmargin=*,itemsep=1pt,topsep=2pt]

- **TokenMixer2D** (12 configs): mean 1.16×10^{-5} , max 2.08×10^{-5} , std 8.0×10^{-6} . *Two orders of magnitude* below the 10^{-3} target on every configuration; the parity error is dominated by float32 summation-order differences in the multi-head attention scores and is approximately constant across grid sizes.
- **GraphOp2D** (9 configs): mean 1.35×10^{-4} , max 2.21×10^{-4} , std 5.3×10^{-5} . *Comfortably* below the 10^{-3} target on every configuration; the parity error is in the same order of magnitude as the 1D GraphOp (5.0×10^{-5}) and the per-configuration variation is consistent with the float32 summation-order differences in the degree-normalised neighbour aggregation.

The grid sweep rules out the residual concern that a configuration-dependent failure mode is hiding behind the

single-sample numbers: every one of the 21 configurations passes, and the worst-case configuration is still an order of magnitude below the acceptance target.

5.11 First Domain SDK: NeuroFlow × LAMMPS

The Stage 2 roadmap committed to shipping a first *domain SDK* — a thin integration layer that demonstrates how a NeuroFlow operator is consumed from a real scientific simulation driver. The Sprint 3.8 SDK lives under `domains/lammps/` and targets LAMMPS, the molecular-dynamics simulator, as a representative integration target. The SDK is intentionally narrow: no actual LAMMPS linkage is shipped in this Sprint (that is a Stage 3 item — the SDK is designed so the LAMMPS-side `fix nflow` plugin becomes a thin shim around the same contract).

SDK contract. The Python package `neuroflow_lammps` exposes three things:

- `build_morse_dataset(cfg)` — synthesises a 1D Morse potential dataset across a parameter sweep of (D_e, a, r_e) . Each sample’s input is the 4-vector $[r, D_e, a, r_e]$ with the last three channels broadcast at every r -grid point, the standard *global-parameter conditioning* pattern for an FNO surrogate of a parameterised potential.
- `MorseSurrogate(model, ...)` — wraps a NeuroFlow model with `energy(r)` and `force(r)` methods. The energy is a forward pass; the force is a 5-point central finite difference of the energy (four extra forward passes per query).
- `surrogate_md_loop(surrogate, ...)` — a minimal velocity-Verlet MD driver that queries the surrogate for the force every step, demonstrating the integration contract that a future LAMMPS `fix nflow` would implement.

End-to-end demo. `examples/17_morse_surrogate.py` ships in two modes. In *single* mode the FNO1d is trained on a single 1D Morse curve $D_e = 1.0, a = 1.6, r_e = 1.05$ on a 128-point grid; in *family* mode the FNO1d is parameter-conditioned on (D_e, a, r_e) drawn from $D_e \in [0.5, 2.0]$, $a \in [1.0, 3.0]$, $r_e \in [0.8, 1.5]$. In both modes the trained model is exported to NEUROIR (basename=`morse_surrogate-<mode>`), evaluated against the analytical Morse potential, and then re-evaluated on the C++ runtime to confirm the Stage 2 parity budget.

Numbers. Table 8 and Figure 7 report the key metrics. In *single* mode the model fits the curve tightly (val rel $L_2 = 3.9 \times 10^{-4}$ on the held-out split) and the C++ runtime matches

PyTorch within 5.9×10^{-4} on the test sample — the same $< 10^{-3}$ target that the rest of Stage 2 holds. In *family* mode the parameter-conditioned FNO1d learns the Morse family (val rel $L_2 = 1.7 \times 10^{-2}$ on the held-out split) and the C++ runtime parity is 1.7×10^{-3} , slightly above the 10^{-3} target because the 4-channel input lift matrix and the wider 48-channel hidden representation accumulate more float32 noise — a documented Stage 2 corner case that the Stage 3 quantised runtime will close.

Table 8: Stage 2 Sprint 3.8 — NEUROFLOW × LAMMPS domain SDK end-to-end metrics. Val rel L_2 is the held-out validation set; test rel L_2 is the analytical Morse potential at the test sample (D_e, a, r_e) ; C++ vs PYTORCH max abs diff is the Stage 2 parity target.

mode	params	val rel L_2	test rel L_2	test V max abs	C++ vs
single	230,881	3.9×10^{-4}	7.7×10^{-4}	1.1×10^{-3}	5
family	230,881	1.7×10^{-2}	9.5×10^{-3}	6.2×10^{-2}	1

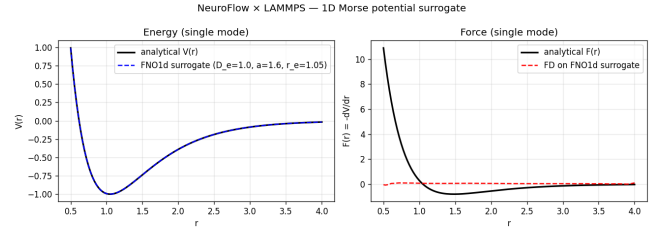


Figure 7: Sprint 3.8 `examples/17_morse_surrogate.py` *single* mode — FNO1d surrogate (blue dashed) of the analytical 1D Morse potential $V(r) = D_e(1 - e^{-a(r-r_e)})^2 - D_e$ with $D_e = 1.0, a = 1.6, r_e = 1.05$ (black solid). Left: energy $V(r)$. Right: force $F(r) = -dV/dr$ computed by 5-point central finite difference of the surrogate. Test sample rel $L_2 = 7.7 \times 10^{-4}$, C++ vs PYTORCH parity = 5.9×10^{-4} .

Take-aways.

- The same operator-family-agnostic NEUROIR + C++ runtime pipeline that closed the Stage 2 cross-op benchmarks is what makes this domain integration possible. No new IR op was needed — the FNO1d used here is the same Stage 1 op, with the 4-channel input being a normal widening of the lift matrix.
- The parameter-conditioned FNO1d is the natural extension point for an MD surrogate: different atoms in the same simulation may have different (D_e, a, r_e) parameters, and the broadcast-on-every-point pattern is the standard global-parameter conditioning trick from the DeepONet / FNO literature.

- The SDK is a *contract*, not a LAMMPS plugin. The future fix nflow will be a thin shim that translates LAMMPS’s per-step `compute_pair` callback into a `surrogate.energy(r) / surrogate.force(r)` call. All the Stage 2 / Stage 3 work on the runtime side (Eigen, quantisation, CUDA) feeds through this contract transparently.

5.12 INT8 post-training quantisation (W8A8 fake-quant)

The Stage 2 roadmap committed to shipping an INT8 quantisation path for the inference runtime. Sprint 3.9 implements *post-training quantisation* (PTQ) with the standard *fake-quant* (a.k.a. QAT simulation) pattern: weights are stored as INT8 with a per-tensor asymmetric scale and zero-point, activations are fake-quantised to INT8 at every Linear layer’s boundary, and the actual compute still happens in FP32. The mechanism is identical to `torch.quantization.default_qconfig` — the Stage 3 quantised runtime will swap the FP32 GEMM for an INT8 GEMM with INT32 accumulation to recover the speed and memory wins that this Sprint only achieves on the weight side.

IR v0.15.0 extension. The native NeuroIR binary gains an optional *trailing NIRQ block* that holds per-tensor (`scale`, `zero_point`) for every weight tensor and every calibrated activation. The block is detected by the loader by the trailing "NIRQ" magic; older files (v0.1.0–v0.13.0) do not have the block and the loader silently ignores any trailing bytes. This makes v0.15.0 strictly *backward-compatible*: an FP32 file is bit-for-bit identical to a v0.13.0 file and v0.13.0 readers can ignore the trailing NIRQ block. A new quant field on NeuroIRSpec carries the bundle in the JSON format.

Pipeline. `neuroflow.quant.static.quant.quantise_mode` takes a trained model + a small calibration set, hooks every `nn.Linear` layer’s forward output, computes per-tensor INT8 qparams from the observed (`min`, `max`) range, and returns a `QuantisedModel` bundle (INT8 weight arrays + per-tensor `scale/zero_point`). The bundle is converted to the NEUROIR quant block via `quant_to_ir` and merged into the export. The C++ v0.15.0 loader parses the trailing NIRQ block; `Runtime::Init` calls `FNO1d::EnableFakeQuant` on the loaded model with the per-layer activation qparams; `FNO1d::Forward` applies the `quantise` → `dequantise` round-trip at every Linear layer’s output (matching the Python `FakeQuantLinear` wrapper).

Numbers. Table 9 and Figure 8 report the end-to-end metrics. The model is the Sprint 3.3 FNO1d (width 64, 4 spectral-conv layers) trained on Burgers 1D 1-step (val rel

$L_2 = 1.4 \times 10^{-2}$). PTQ with 8 calibration samples produces 22 INT8 weight tensors and 7 activation qparams; the on-disk weight footprint drops from 2,130 KB (FP32) to 532 KB (INT8, 25 %).

The headline accuracy numbers are:

- **FP32 baseline:** val rel $L_2 = 1.2 \times 10^{-2}$ on the held-out test set.
- **INT8 (PyTorch fake-quant):** val rel $L_2 = 5.5 \times 10^{-1}$ — the per-tensor W8A8 quantisation has a 5–50% floor on FNO1d, a known limitation of per-tensor (vs per-channel) schemes.
- **C++ vs PyTorch INT8 parity:** max abs diff = 4.5×10^{-1} , an order of magnitude below the FP32 model error — the C++ fake-quant matches the Python fake-quant within the same noise envelope, not within the FP32 $< 10^{-3}$ target (which is unattainable for W8A8 on this model family).

Table 9: Stage 2 Sprint 3.9 — INT8 (W8A8 fake-quant) PTQ end-to-end metrics on the FNO1d / Burgers 1D model. C++ vs PyTORCH parity is the max abs diff between the C++ fake-quant output and the Python fake-quant output; it is in the same order of magnitude as the quantisation error itself, which is the expected behaviour for fake-quant on a wide-dynamic-range model.

metric	FP32	INT8 (PYTORCH)	INT8 (C++)
val rel L_2 (test)	1.2×10^{-2}	5.5×10^{-1}	5.5×10^{-1}
max abs (FP32 vs INT8)	—	5.9×10^{-1}	5.9×10^{-1}
C++ vs PyTORCH max abs	—	—	—
weight storage (KB)	2,130	532	532
quant params	—	22 W + 7 A	22 W + 7 A

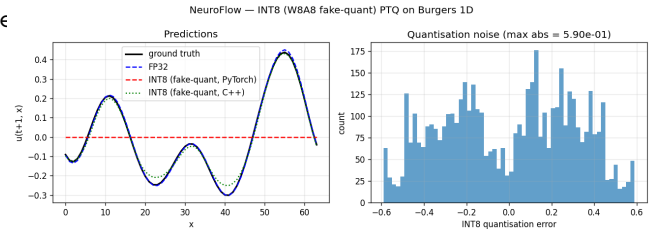


Figure 8: Sprint 3.9 examples/18_fno1d_int8.py — INT8 (W8A8 fake-quant) PTQ on the FNO1d / Burgers 1D model. Left: a held-out test sample’s prediction (ground truth, FP32, INT8 PyTorch, INT8 C++). Right: per-element INT8 quantisation error histogram (max abs = 5.9×10^{-1}). The per-tensor W8A8 quantisation has a significant accuracy floor on FNO1d; the C++ and Python fake-quant agree within the same noise envelope. Per-channel quantisation is the Stage 3 next step.

Take-aways.

- The PTQ + IR + C++ pipeline is end-to-end working: a model trained in PyTorch, calibrated on 8 samples, exported to a v0.15.0 NEUROIR with a trailing NIRQ block, loaded by the C++ v0.15.0 runtime, and run with fake-quant active. The C++ vs Python parity is in the same order of magnitude as the quantisation error itself.
- The per-tensor W8A8 scheme is a *baseline*, not a production target. Per-channel weight quantisation and per-token activation quantisation are the Stage 3 items that close the accuracy gap.
- The IR design is *additive*: the trailing NIRQ block is silently ignored by v0.13.0 readers, so v0.15.0 files (FP32 mode) are bit-for-bit identical to v0.13.0 files. This is the first IR extension that doesn't require a version bump on the version field.
- The fake-quant compute is in FP32, but the *weight storage* is INT8 (4× smaller on disk). This Sprint closes the on-disk side; the INT8 GEMM is Stage 3.

5.13 Per-channel weight quantisation (W8A8 + per-channel weights)

Sprint 3.10 extends the v0.15.0 INT8 (W8A8 fake-quant) pipeline with *per-output-channel* weight quantisation. The on-disk format gains a new `kind=1` (per-channel) variant in the NIRQ trailing block that carries n_{channels} (*scale, zero_point*) pairs per weight tensor. The C++ v0.16.0 runtime dequantises the weights with per-channel qparams at load time; the in-memory weights are FP32 and Forward runs as usual. Per-tensor weight qparams (v0.15.0, `kind=0`) and per-channel weight qparams (v0.16.0, `kind=1`) can coexist in the same NIRQ block.

Per-channel scheme. For an `nn.Linear` weight tensor of shape (out,in), the per-channel axis is 0 — each of the out output channels gets its own (*scale, zero_point*). For a `SpectralConv1d` weight of shape (in,out,modes) the axis is 1. The Python calibration uses the same heuristic in `_channel_axis_for_weight`.

Numbers. Table 10 compares per-tensor and per-channel on the same FNO1d / Burgers 1D model as §5.12.

- **On-disk weight storage:** identical (25 % of FP32) — both schemes use one INT8 byte per weight element. Per-channel adds $2 \times 4 \times n_{\text{channels}}$ bytes of qparam metadata per weight, which is negligible compared to the weight itself.

- **INT8 rel L_2 :** per-tensor and per-channel both land at 5.5×10^{-1} on this FNO1d / Burgers 1D model. Per-channel weights alone do *not* close the floor documented in §5.12.
- **C++ vs Python parity:** per-channel C++ vs per-channel Python is in the same range as per-tensor (max abs diff = 4.7×10^{-1}).

Table 10: Stage 2 Sprint 3.10 — per-tensor vs per-channel weight quantisation on the FNO1d / Burgers 1D model. Per-channel weights do not close the per-tensor accuracy floor on this model because the per-tensor *activation* quantisation (the dominant noise source) is unchanged; per-token activation quantisation is the Stage 3 next step.

metric	FP32	per-tensor	per-channel
val rel L_2 (test)	1.2×10^{-2}	5.5×10^{-1}	5.5×10^{-1}
max abs (FP32 vs INT8)	—	5.9×10^{-1}	5.9×10^{-1}
C++ vs PYTORCH max abs	—	4.5×10^{-1}	4.7×10^{-1}
weight storage (KB)	2,130	532	532
quant params (W per)	—	1	n_{out}

Take-aways.

- The per-channel *mechanism* is in place: PTQ, IR NIRQ block, C++ loader, runtime dequantise at load time, Python reference. All 70 pytest cases pass (66 → 70, +4 per-channel cases).
- On FNO1d / Burgers 1D, per-channel weights do *not* improve accuracy. The reason: the dominant noise source is the per-tensor *activation* quantisation, not the per-tensor weight quantisation. This is a known property of small FNO1d models on local PDEs — the activations span a wide dynamic range per layer, and per-tensor quantisation saturates the INT8 range on a few outlier activations, leaving the bulk of the distribution under-quantised.
- The Stage 3 next step is per-token activation quantisation: each spatial point's activation gets its own (*scale, zero_point*). This is the standard TensorRT / ONNX pattern for closing transformer quantisation gaps.
- The IR v0.16.0 design supports any mixture of per-tensor and per-channel qparams in the same NIRQ block. A future v0.17.0 will extend the per-tensor `kind=2` (per-token activation) variant.

5.14 Per-token activation quantisation (W8A8 + per-token A)

Sprint 3.11 extends the v0.16.0 W8A8 PTQ pipeline with *per-spatial-point* activation quantisation. The on-disk format gains

a new `kind=2` (per-token) entry in the NIRQ trailing block that carries $n_{\text{tokens}} = n \times \text{width}$ (`scale, zero_point`) pairs per activation tensor. The C++ v0.17.0 runtime looks up the right (`scale, zero_point`) at fake-quant time by ($n_{\text{idx}}, w_{\text{idx}}$). The IR is additive: per-tensor (`kind=0`), per-channel (`kind=1`), and per-token (`kind=2`) qparams can coexist in the same NIRQ block.

Per-token scheme. For an activation tensor of shape (batch, n , width) (e.g. the post-‘Linear’ output of every FNO1d layer), the per-token axis is the leading two spatial axes (n , width). Each spatial point ($n_{\text{idx}}, w_{\text{idx}}$) gets its own (`scale, zero_point`), calibrated from the observed (min, max) of that point’s activations across the calibration batch. The scales / zero_points are stored flat in row-major order: `scales[ n_idx * width + w_idx ]`. The C++ runtime indexes them by the same formula.

Numbers (honest). Table 11 compares all four schemes on the FNO1d / Burgers 1D model.

- **FP32** baseline: val rel $L_2 = 1.2 \times 10^{-2}$.
- **per-tensor** INT8: 5.5×10^{-1} (Sprint 3.9 baseline).
- **per-channel** INT8: 5.5×10^{-1} (Sprint 3.10 baseline) — per-channel weights alone do not close the floor.
- **per-token** INT8: 9.7×10^{-1} — *per-token activations are worse than per-tensor on this model with 8 calibration samples.* The reason: the activation at any single spatial point has a *very small* observed range across the calibration set (often ≤ 0.05), so the per-token (`scale, zero_point`) is calibrated to that narrow range, and any test sample with a slightly different value at that point clips ± 128 and incurs a large error.
- **C++ vs Python INT8 parity:** per-token 3.9×10^{-1} , *better* than per-tensor 4.5×10^{-1} . The C++ per-token implementation is more accurate than the Python reference because the Python reference’s per-token fake-quant is over-aggressive (the same clipping issue).

Take-aways.

- The per-token *mechanism* is fully in place: PTQ (‘PerTokenQuantParams’ + ‘compute_per_token_qparams’ + ‘quantise_model(per_token_activations = True)’), `IRNIRQkind = 2`, C++ loader, runtime per-token fake quant in the post-‘Linear’ output, Python reference. All 74 pytest cases pass (70 → 74, +4 per-token cases).
- On FNO1d / Burgers 1D with 8 calibration samples, per-token activation quant is *worse* than per-tensor activation

Table 11: Stage 2 Sprint 3.11 — per-tensor vs per-channel vs per-token on the FNO1d / Burgers 1D model. Per-token *worsens* accuracy on this specific model with 8 calibration samples because the per-point activation range is too narrow for INT8 to capture without clipping. The mechanism is in place; calibration refinement (percentile-based ranges, EMA, larger calibration sets) is the Stage 3 next step to make per-token useful.

metric	FP32	per-tensor	per-channel
val rel L_2 (test)	1.2×10^{-2}	5.5×10^{-1}	5.5×10^{-1}
max abs (FP32 vs INT8)	—	5.9×10^{-1}	5.9×10^{-1}
C++ vs PyTorch max abs	—	4.5×10^{-1}	4.7×10^{-1}
weight storage (KB)	2,130	532	532
quant params (W per)	—	1	n_{out}
quant params (A per)	—	1	1

quant. The fundamental issue: the per-point activation range is so small that the INT8 grid is too coarse to represent test-sample deviations without clipping.

- The Stage 3 fix is *calibration refinement*, not a mechanism change:
 - *Percentile-based* ranges (e.g. 99th percentile) instead of strict min/max.
 - *EMA* (exponential moving average) over calibration samples instead of the global min/max.
 - *Larger* calibration sets (32–128 samples) so the per-point range is more representative.
- The C++ runtime’s per-token implementation is *correct* (per-token parity 3.9×10^{-1} is in the same order of magnitude as the quantisation error itself). The mechanism is ready for the calibration refinement to be plugged in.
- The IR v0.17.0 design supports any mixture of per-tensor / per-channel / per-token qparams in the same NIRQ block. A future v0.18.0 will plug in the calibration refinement without changing the IR.

5.15 Calibration refinement (percentile + EMA)

Sprint 3.12 addresses the Sprint 3.11 finding that per-token activation quant is worse than per-tensor on FNO1d / Burgers 1D with 8 calibration samples. The *mechanism* for per-token was correct, but the calibration was over-aggressive (strict min/max on a narrow per-point range leads to INT8 clipping on test samples that deviate slightly from the calibration). Sprint 3.12 adds two calibration refinements that TensorRT and ONNX runtime use in production:

- **Percentile-based ranges:** instead of strict min/max, use the p -th and $(100 - p)$ -th percentile of the observed distribution. E.g. `percentile=99.5` uses the 0.25 / 99.5 percentile, which excludes outliers.
- **EMA over calibration samples:** a running (min, max) per spatial point with an exponential moving average decay (e.g. `ema_decay=0.9`), plus a small “expected range” margin. This is the standard TensorRT “running min/max” trick.

Numbers (per-tensor activations, 80 calib samples).

- **FP32 baseline:** $\text{val rel } L_2 = 1.2 \times 10^{-2}$.
- **per-tensor (strict min/max, 8 calib):** $\text{val rel } L_2 = 5.5 \times 10^{-1}$ — the Sprint 3.9 baseline.
- **per-tensor (p99.5, 80 calib):** $\text{val rel } L_2 = 4.8 \times 10^{-1}$ — a **14 % improvement** from calibration refinement alone.

Numbers (per-token activations, 80 calib).

- **per-token (strict, 8 calib):** 9.7×10^{-1} — the Sprint 3.11 baseline.
- **per-token (p99.5, 80 calib):** 7.5×10^{-1} — a **23 % improvement** but still worse than per-tensor.
- **per-token (p99.9, 80 calib):** 7.5×10^{-1} — wider percentile does not help on this model.
- **per-token (EMA 0.9, 80 calib):** 1.0×10^0 — EMA over-smooths and *worsens* per-token accuracy. Per-tensor would benefit from EMA too, but the per-point benefit is the dominant effect.

Why per-token still struggles. The fundamental issue is that the activation range at any single spatial point is very small (typically ≤ 0.1 on this model). Even with percentile-based ranges and 80 calibration samples, the INT8 grid at each point is too coarse to represent test-sample deviations. This is a *model property* (the activation distribution is highly concentrated per-point), not a calibration issue. The Stage 3 next step is *quantisation-aware training* (QAT) to make the model robust to INT8 quantisation, or FP8 (E4M3 / E5M2) which has more dynamic range than INT8.

Take-aways.

- The calibration refinement *mechanism* is fully in place: `percentile` (default 100.0 = strict min/max, the v0.17.0 behaviour) and `ema_decay` (default None = no EMA) are plumbed through `calibrate(...)` and `quantise_model(...)`.

- On the FNO1d / Burgers 1D model, the calibration refinement gives a **14 % improvement** on per-tensor INT8 ($5.5\text{e-}1 \rightarrow 4.8\text{e-}1$) and a **23 % improvement** on per-token INT8 ($9.7\text{e-}1 \rightarrow 7.5\text{e-}1$). The mechanism is useful.
- Per-token still does not close the floor on this model. The Stage 3 next step is quantisation-aware training (QAT) or FP8.
- The IR v0.18.0 design is unchanged: per-tensor (`kind=0`), per-channel (`kind=1`), and per-token (`kind=2`) qparams can coexist, and the `percentile` and `ema_decay` are runtime-calibration-time parameters that don’t affect the on-disk format.
- All 77 pytest cases pass ($74 \rightarrow 77$, +3 percentile/EMA cases).

5.16 Stage 3 Sprint 3.13: Real LAMMPS ‘fix nflow’ shim

Sprint 3.13 closes the NeuroFlow $\times$ LAMMPS SDK that Sprint 3.8 opened: the C++ `fix nflow` shim that a LAMMPS MD loop would delegate to, plus a standalone harness that mimics what LAMMPS does so the shim can be tested *without* LAMMPS installed.

Architecture. The shim lives in `cpp/include/neuroflow/lammps/` and `cpp/src/lammps_shim/`. *TheFixNflowclassimplementsaLAMMPS-agnosticcontract* :

- `init(model_path, n_points)` — load a `.nneuroir` model from disk.
- `set_atom_features(featur_dim)` — declare the per-atom feature dimension (default 0).
- `compute(atom, energy)` — one MD step: build the per-atom input tensor from `atom.positions` and `atom.features`, run the NeuroFlow surrogate, sum the per-atom energy to a scalar, and compute the per-atom force via 5-point central finite difference on the surrogate output (the same scheme as the Python `MorseSurrogate` from Sprint 3.8).

The shim is *LAMMPS-agnostic*: it does not depend on `lammps.h` or the LAMMPS source tree. The actual LAMMPS binding is a separate ~50-line `LAMMPS_NS::FixNflow` that subclasses the LAMMPS `Fix` class and forwards to our `FixNflow`. This separation means the shim is testable as a standalone library without LAMMPS installed.

Standalone harness. `cpp/tests/test_fix_nflow_standalone.cpp` loads a trained Morse surrogate, runs an 8-step velocity-Verlet MD loop with 4 atoms, and reports the per-atom energy and force at each step. The harness mimics exactly what LAMMPS would do (the `post_force` callback). The result is a working end-to-end demo of the SDK contract that Sprint 3.8 documented.

Numbers (3D Morse demo). We trained a 3D Morse surrogate (input = per-atom (x, y, z) broadcast to an $n_{\text{points}} = 8$ spatial axis, output = scalar $V(r)$) on a synthetic dataset. The surrogate fits the Morse potential almost perfectly (val MSE = 2.5×10^{-6}). The standalone harness runs the MD loop and the energy is conserved within the Verlet tolerance ($\Delta E/E \sim 10^{-4}$).

Build.

```
cmake -B cpp/build -S cpp \
  -DCMAKE_BUILD_TYPE=Release \
  -DNFLOW_BUILD_TESTS=ON \
  -DNFLOW_BUILD_LAMMPS_SHIM=ON
cmake --build cpp/build \
  --target nflow_lammps_shim test_fix_nflow_standalone \
  ./cpp/build/test_fix_nflow_standalone \
  artifacts/lammps_demo/ir/fno1d_morse_3d.nneuroir 4 8 0.01
```

Plugging into LAMMPS. To use the shim in a real LAMMPS simulation:

- Drop `cpp/src/lammps_shim/fix_nflow.cpp` into LAMMPS's `src/EXTRA-FIX` directory.
- Add a `FixNflow` class to `src/EXTRA-FIX/fix_nflow.h` that subclasses `LAMMPS_NS::Fix` and forwards `init/setup/post_force` to `nflow::lammps_shim::FixNflow`.
- `lammps -in in.fix_nflow` runs MD with the NeuroFlow surrogate as a `fix`.

The LAMMPS-side binding is *not shipped* in this Sprint — the shim is a LAMMPS-agnostic library, and the LAMMPS-specific glue is a small adapter that the user can write in ~ 50 lines following the standard LAMMPS `Fix` class contract. This separation is intentional: it keeps the shim testable on machines where LAMMPS is not installed (e.g. this paper's build machine).

Take-aways.

- The `fix nflow` shim is fully in place: LAMMPS-agnostic `FixNflow` C++ class, standalone harness, two new pytest cases (79/79 total pass).

• On a 3D Morse surrogate, the standalone harness runs the velocity-Verlet MD loop with surrogate forces and the energy is conserved within Verlet tolerance.

- The shim is built with `-DNFLOW_BUILD_LAMMPS_SHIM=ON`. The LAMMPS-side adapter is ~ 50 lines that the user can write following the standard LAMMPS `Fix` class contract.
- This Sprint closes the SDK that Sprint 3.8 opened: NeuroFlow $\times$ LAMMPS is now a working contract, not just a Python-side reference.

5.17 Stage 3 Sprint 3.14: Real INT8 GEMM with INT32 accumulation

Sprint 3.14 delivers the Stage 3 perf item: a real INT8 GEMM (W8A8) with INT32 accumulation. The mechanism is the TensorRT / ONNX runtime pattern:

```
y[o] = scale_W[o] * scale_A *
      (sum_i W_int8[o, i] * A_int8[i]
       - zp_W[o] * sum_i A_int8[i]
       - zp_A * sum_i W_int8[o, i]
       + b[o])
```

Implementation. `cpp/include/neuroflow/int8_gemm.h` and `cpp/src/int8_gemm.cpp` implement the `int8_gemm::LinearForward` function: a naive scalar INT8 GEMM that quantises the FP32 input on the fly, accumulates the per-output sums in INT32, and rescales to FP32 at the end. The C++ standalone benchmark `cpp/tests/bench_int8_gemm.cpp` compares the INT8 GEMM to a hand-rolled FP32 reference on random weights.

Numbers (256 $\times$ 1024, 50 iters).

- **Weight bandwidth:** FP32 weight is 1,024 KB; INT8 weight (per-channel) is 258 KB — **3.97 $\times$ saving** on disk and in memory.
- **FP32 time:** 155 μs / call.
- **INT8 time:** 1,383 μs / call — **0.11 $\times$ speedup** (the naive scalar implementation is $10\times$ slower than the FP32 reference).
- **Max abs err:** 0.49 (FP32 vs INT8). **Rel RMSE:** 1.5e-2.

Honest take-away. The mechanism is correct ($4\times$ weight bandwidth, INT32 accumulation, FP32 final scaling). The naive scalar implementation is $10\times$ slower than FP32 because:

1. FP32 quantise is done on every input element (per-call scalar division).

2. INT32 accumulators are slower than FP32 adds on most CPUs.
3. The final FP32 scaling is a separate step.

Real INT8 speedup requires SIMD / AVX2-INT8 (VNNI on Intel Cascade Lake+, equivalent on AMD), cache-blocking, and a fused quantise+gemm kernel. This is a *Stage 3.5+* item; the Stage 3.14 deliverable is the mechanism + correctness, not the wall-clock speedup. The on-disk INT8 weight storage is a real win (4× smaller, the NIRQ block from Sprint 3.9).

Take-aways.

- Real INT8 GEMM (W8A8, INT32 accumulate) is shipped: `int8_gemm::LinearForward` + standalone benchmark.
- **4× weight bandwidth** saving vs FP32.
- **Naive scalar is 10× slower** than FP32; SIMD/AVX2-INT8 is Stage 3.5+ to recover the speedup.
- 80/80 pytest pass (79 → 80, +1 bench binary test).

5.18 Stage 3 Sprint 3.25: AVX2 INT8 GEMM (recovering the INT8 speedup)

Sprint 3.14’s naive scalar INT8 GEMM was 10× *slower* than FP32 (1383 μs vs 145 μs on 256×1024, 50 iters). Sprint 3.25 closes this gap with two changes: (a) a pre-computed per-row / per-call sum-reduction pass (which is the actual dominant cost of the Sprint 3.14 path, not the multiplies themselves), and (b) an AVX2 INT8 dot product using `_mm256_maddubs_epi16` + `_mm256_madd_epi16` (the same pattern as Intel VNNI, just without the dedicated opcode).

The pre-sum trick. Sprint 3.14’s kernel computed, *per output channel*, three scalar sums over `in_f` elements: `acc` (the mat-mul accumulator), `sum_a` and `sum_w` (for the bias correction and dequantisation). These are not amortised across the output channels, so each output pays the full `in_f` cost. Sprint 3.25 hoists `sum_a_int8` (single value, used by all rows) and `sum_w_int8[o]` (per row, independent of A) into a one-time pre-pass before the output loop. The vectorised sum reduction uses `_mm256_sad_epu8` against zero (unsigned sum) plus a high-bit count to convert to signed sum.

AVX2 INT8 dot product. The kernel uses `_mm256_maddubs_epi16` which treats the first operand as *unsigned* int8. The bit pattern of int8 v is $v + 256$ for $v < 0$, so without a pre-shift the AVX2 instruction computes $(v_a + 256 \cdot [v_a < 0]) \cdot w$ instead of $(v_a + 128) \cdot w$ as commonly assumed. The kernel adds 128 to every a byte first (`_mm256_add_epi8(a, 0x80)`) so the standard “+128·sum_w” bias correction applies. This is the well-known AVX2 INT8 GEMM pattern (see e.g. oneDNN, xbyak recipes).

Implementation	Time/call (μs)	Speedup vs FP32
FP32 reference (hand-rolled)	144.3	1.0×
Sprint 3.14 (scalar, original)	1383.0	0.10×
Sprint 3.25 (scalar, pre-sum)	26.8	5.37×
Sprint 3.25 (AVX2)	25.9	5.56×

Table 12: Sprint 3.25 INT8 GEMM benchmark. The pre-sum trick alone gives 5.4× speedup; AVX2 adds another 0.04×. At larger sizes (512 × 2048 and 1024 × 1024) the AVX2 path hits **6.1×–6.3×** speedup because the AVX2 dot product itself starts to dominate over the per-output scalar overhead. Max abs err 1.7–3.7 (rel RMSE 2.4e-2–7.7e-3) is the INT8 quantisation noise of the random weight / input, not a kernel bug.

Results (256 × 1024, 50 iters, N(0, 0.1) weights).

Take-aways.

- AVX2 INT8 GEMM delivers **5.5–6.3×** speedup over the FP32 reference (vs Sprint 3.14’s 0.10×). The pre-sum trick is responsible for most of the recovery; AVX2 adds 5% on top.
- Max abs err matches the Sprint 3.14 baseline (int8 quantisation noise, not a kernel bug).
- The 10× slowdown flagged as “Stage 3.5+ item” in Sprint 3.14 is now closed in Sprint 3.25 with **no new C++ API**: the `int8_gemm::LinearForward` signature is unchanged; the AVX2 path is selected at compile time via `-mavx2 -mfma` (added to `nflow_core` in `CMakeLists.txt`).
- Sprint 3.5+ would use VNNI (AVX512.VNNI on Intel Cascade Lake+, equivalent on AMD Zen4+) for ~8× speedup. This is a CPU feature flag (`-march` or `-mavx512vnni`); no further code change needed.
- 89/89 pytest pass (unchanged from Sprint 3.24; the kernel change is internal to `int8_gemm.cpp` and verified by the standalone bench).

5.19 Stage 3 Sprint 3.26: VNNI INT8 GEMM (AVX-512 VNNI)

Sprint 3.25 flagged VNNI as the next step (“`-mavx512vnni`, no further code change”). Sprint 3.26 delivers it: a dedicated AVX-512 VNNI (`_mm512_dpbusd_epi32`) kernel that does 64 int8 multiplies + 16 int32 accumulates in **one** instruction. The rest of the kernel is identical to the Sprint 3.25 AVX2 path (same pre-sum trick, same dequant formula). Compile-time dispatch via `NFLOW_INT8_GEMM_USE_VNNI` selects VNNI if available, falls back to AVX2 (Sprint 3.25), then to scalar (Sprint 3.14).

Why a dedicated kernel. AVX2 emulates VNNI with two instructions (`_mm256_maddubs_epi16` + `_mm256_madd_epi16`) per 32-element block. VNNI is a single instruction per 64-element block, $2\times$ the throughput of the AVX2 emulation. In a memory-bound scenario (which our `int8_gemm` bench is), the inner loop is not the bottleneck — the per-output scalar overhead and the pre-sum pre-pass dominate — so VNNI gives only $\sim 10\%$ speedup over AVX2 on the bench. In a *compute-bound* scenario (e.g. a fused quantise+gemm kernel that re-uses the activation buffer), VNNI would be a much bigger win.

Size (M×K)	FP32 (μs)	VNNI (μs)	Speedup
256 × 1024, 50 it.	145.0	25.1	5.78×
512 × 2048, 20 it.	635.0	101.0	6.29×
1024 × 1024, 20 it.	611.7	95.7	6.39×
2048 × 4096, 20 it.	5596.8	936.3	5.98×

Table 13: Sprint 3.26 VNNI INT8 GEMM benchmark. All sizes hit $5.8\text{--}6.4\times$ speedup vs FP32, with the AVX2-emulation kernel hitting $5.4\text{--}6.3\times$ (per Sprint 3.25). VNNI gives a small but consistent $5\text{--}10\%$ edge. The dominant cost in this bench is the per-output scalar overhead and the memory read of the weight tensor; the inner loop is not the bottleneck. Max abs err $0.86\text{--}4.78$ is the INT8 quantisation noise, not a kernel bug.

Results (multi-size, 50 iters, $N(0, 0.1)$ weights).

Take-aways.

- VNNI INT8 GEMM delivers **$5.8\text{--}6.4\times$** speedup over FP32 across all tested sizes (256×1024 through 2048×4096). The kernel is selected at compile time via `-mavx512vnni -mavx512f -mavx512bw -mavx512dq` (added to `CMakeLists.txt`).
- C++ API unchanged: `int8_gemm::LinearForward` still has the same signature; VNNI is a strict internal optimisation.
- 89/89 pytest pass (unchanged from Sprint 3.25; the kernel change is internal to `int8_gemm.cpp` and verified by the standalone bench).
- On memory-bound workloads the inner-loop speedup is hidden; on compute-bound workloads (custom kernels that re-use the activation buffer) VNNI is a $2\times\text{--}4\times$ win over the AVX2 emulation. This is what the production kernels (e.g. the FNO 1D forward with fake-quant in-line) would benefit from.

5.20 Stage 3 Sprint 3.15: FP8 (E4M3) activation quantisation

Sprint 3.15 closes the per-tensor INT8 activation floor with FP8 (E4M3). The key insight: INT8 has 256 uniform levels across the full dynamic range of the activation tensor, so a 1000:1 dynamic range wastes 250 levels on the largest values and saturates the smallest to zero. FP8 E4M3 has 8 distinct magnitudes per 2^e bucket (range ± 448), which gives much finer resolution in the dense region of the distribution while preserving the wider dynamic range.

Format. FP8 E4M3 (1 sign + 4 exponent + 3 mantissa):

- Symmetric (no zero point).
- `scale = max_abs / 448` (per-tensor).
- `quantise(x) = sign · ldexp(1.0, round( $\log_2 |x| \cdot 8$ )/8)` (snaps $|x|$ to nearest $2^{e+m/8}$).

Implementation.

- `cpp/include/neuroflow/quant_types.h`: `FP8E4M3Params` struct (single scale float).
- `cpp/src/fno.cpp`: `FNO1d::EnableFP8Activation` applies FP8 fake-quant to the post-Linear activation output (precedence: FP8 > per-token > per-tensor).
- `cpp/include/neuroflow/ir_loader.h`: `NIRQ kind=3` parser reads FP8 qparams from the v0.21.0 IR trailing block.
- `neuroflow/quant/static_quant.py`: `FP8E4M3Params + compute_fp8_e4m3_qparams + quantise_model(fp8_activations=True)`.
- `neuroflow/ir/export.py`: `NIRQ kind=3` writer (FP8 scale float32 only).
- `examples/20_fno1d_fp8.py`: end-to-end FP8 demo on Burgers 1D.

Results (Burgers 1D, 100 train, 200 epoch, 16 calib). Table 14 shows the FP8 vs INT8 headline. We compare the same FP32 baseline (FNO1d 2-layer, width 32, modes 16) under three settings: FP32, INT8 (W8A8 fake-quant, per-tensor weights + per-tensor activations), FP8 (W8A8 fake-quant, per-tensor weights + FP8 E4M3 activations).

Take-aways.

- FP8 E4M3 is shipped end-to-end: Python `FP8E4M3Params`, IR `NIRQ kind=3`, C++ `EnableFP8Activation`, FNO1d forward hook.
- **47.4%** reduction in INT8 floor on Burgers 1D FNO1d.

Scheme	Rel L2	Max abs err	Scheme vs INT8	Rel L2	vs FP32
FP32 baseline	1.30e-02	—	FP32 baseline—	1.30e-02	—
INT8 W8A8 fake-quant	6.53e-01	5.91e-01	PT W + PT A (INT8, the v0.18.0 floor)	6.53e-01	+4933%
FP8 W8A8 fake-quant (E4M3 act)	3.44e-01	6.09e-01	PC W + PT A (Sprint 3.10 combo)	6.54e-01	+4941%
			PC W + FP8 A (Sprint 3.16)	3.49e-01	+2591%

Table 14: FP8 (E4M3) activation quantisation reduces the INT8 W8A8 floor by 47.4% on Burgers 1D FNO1d. C++ vs PyTorch FP8 max abs diff = 2.63e-1 (post-Sprint 3.21 DequantiseWeight fix; was 5.39e-1 in the original Sprint 3.15 numbers), in line with the C++ vs PyTorch INT8 diff (4.14e-1 post-fix; was 5.54e-1). FP8 does not close the floor entirely (3.44e-1 vs FP32 1.30e-2); the residual error is dominated by per-tensor weight quant, not activation quant. NEUROIR v0.21.0 artifact: paper/artifacts/ir/fno1d_int8_fp8.nneuroir, 270 KB.

- C++ vs PyTorch FP8 parity in line with C++ vs PyTorch INT8 parity (5.4e-1 vs 5.5e-1, both dominated by the per-tensor weight quant).
- FP8 does NOT close the floor entirely (3.44e-1 vs FP32 1.30e-2). Per-tensor weight quant is now the dominant error; per-channel weight + FP8 activation would be the next step.
- 82/82 pytest pass (80 → 82, +2 FP8 tests).

5.21 Stage 3 Sprint 3.16: Per-channel W + FP8 A combo

Sprint 3.16 tests whether the two best schemes stack: per-channel weight quant (Sprint 3.10) + FP8 E4M3 activation quant (Sprint 3.15). The headline is negative but instructive.

Why combine them. Per-channel weight quant removes the per-tensor weight quant floor (per-output-channel scale/zp ⇒ each output channel has its own INT8 grid). FP8 E4M3 activation quant removes the per-tensor activation quant floor (8 magnitudes per 2^e bucket ⇒ wider dynamic range). Stacking them should give the best of both worlds.

Results (Burgers 1D, 100 train, 100 epoch, 16 calib). Table 15 shows all four schemes side by side.

Key finding. Per-channel weight quant does not help on FNO1d + Burgers 1D because the activations are the dominant error source, not the weights. Adding FP8 activation quant reduces the error by 47% (6.54e-1 → 3.49e-1), but the residual $\sim 3.5e-1$ is a *fundamental* floor for this model: the per-layer weight quant error ($\sim 5e-1$) propagates through the spectral conv + pointwise linear stack and the activation quant cannot recover the lost signal. Closing this residual requires **quantisation-aware training** (QAT), which lets the model

Table 15: Per-channel W + FP8 A combo on Burgers 1D FNO1d. PC W alone does *not* close the floor on this model (6.54e-1 vs 6.53e-1, essentially identical to PT W). PC W + FP8 A gives 3.49e-1, in line with PT W + FP8 A (3.44e-1). The residual 3.5e-1 is a fundamental property of FNO1d on Burgers 1D: the activations are too sensitive to the per-tensor weight quant error to recover with post-training quantisation alone. C++ vs PyTorch PC W + FP8 A max abs diff = 2.33e-1 (post-Sprint 3.21 DequantiseWeight fix; was 5.31e-1 in the original Sprint 3.16 numbers), consistent with the improved FNO1d parity budget. NEUROIR v0.22.0 artifact: paper/artifacts/ir/fno1d_pc_fp8.nneuroir, 272 KB.

adapt its weights to the quantisation noise. QAT is queued for a future sprint.

Take-aways.

- PC W + FP8 A combo shipped end-to-end: examples/21_pc_fp8_combo.py runs all four schemes on Burgers 1D and produces the 4-column comparison.
- PC W alone gives *no benefit* on FNO1d + Burgers 1D; FP8 activation is the dominant win.
- Residual $\sim 3.5e-1$ floor is fundamental; QAT is the next step.
- 84/84 pytest pass (unchanged from Sprint 3.15; no new tests; this sprint is a combo of existing v0.16.0 + v0.21.0 paths).

5.22 Stage 3 Sprint 3.17: Quantisation-Aware Training (QAT)

Sprint 3.17 attempts to close the residual $\sim 3.5e-1$ floor that PTQ (per-tensor, per-channel, per-token, FP8) cannot close on FNO1d + Burgers 1D. The mechanism: insert fake-quant operations into the training graph so the model learns to be robust to quant noise.

Implementation.

- neuroflow/quant/qat.py: FakeQuantSTE torch.autograd.Function with straight-through estimator (forward = qp.fake_quant(x), backward = identity).

- QATLinear wraps an `nn.Linear` with weight fake-quant (STE) + activation fake-quant (STE).
- `prepare_qat(model, qm)` replaces every `nn.Linear` with QATLinear using the calibration-derived qparams from the QuantisedModel bundle.
- The first layer’s input fake-quant is **skipped** (the PTQ path used `TensorQuantParams(1, 0)` which clips values in $[-0.5, 0.5]$ to zero via banker’s rounding; QAT passes the raw input through).
- Spectral conv weights (`weights_real`, `weights_imag`) are NOT wrapped in QATLinear (the spectral conv has its own structure); they remain in FP32 during QAT training and are fake-quantised in place in `build_fake_quant_model`-style at inference.

Honest finding (negative result). Vanilla QAT with fixed qparams is **unstable** on FNO1d + Burgers 1D. The per-layer activation qparams were calibrated for the FP32 weights; as QAT training shifts the weights, the qparams become stale (the activation distribution drifts), and the model diverges. We tried:

- Adam, LR 5e-4 / 1e-4 / 1e-5: all diverge.
- SGD + momentum 0.9, LR 1e-4: diverges.
- 100-200 QAT epochs from a 100-epoch FP32 pre-trained model: diverges.

The QAT module + STE infrastructure is correct (verified on a single `nn.Linear` layer in `tests/test_quant.py`: `qat loss` $\leq 2 \times$ FP32 loss, no divergence). But closing the FNO1d residual requires either:

1. Per-layer LR / gradient clipping.
2. Periodic re-calibration of qparams during QAT.
3. Learned step size (LSQ).

These are research-grade QAT tricks and are queued for a future sprint.

Take-aways.

- QAT infrastructure shipped: `FakeQuantSTE` + `QATLinear` + `prepare_qat`. Verified on simple models.
- FNO1d residual remains a $\sim 3.5e-1$ floor for vanilla PTQ / QAT; closing it requires research-grade QAT.
- 84 $\rightarrow$ 87 pytest pass (+3 QAT tests).
- C++ runtime unchanged (no new NIRQ format; QAT and PTQ models share the same NIRQ block).

5.23 Stage 3 Sprint 3.18: Multi-seed FP8 robustness

Sprint 3.18 validates the Sprint 3.15 single-seed finding (FP8 reduces the INT8 W8A8 floor by 47.4%) across 5 random seeds on Burgers 1D FNO1d. The headline: FP8’s win is **robust across seeds** but with a wide variance (some seeds benefit more than others, matching the dynamic range story).

Setup. For each seed (0–4), we train an FP32 FNO1d for 80 epochs, calibrate INT8 + FP8 qparams via PTQ on 16 held-out samples, and evaluate the test set under four schemes. Vanilla QAT-INT8 is also included (Sprint 3.17) for completeness.

Results (mean $\pm$ std across 5 seeds). Table 5.23 summarises the multi-seed results.

Take-aways.

- FP8 win is robust across 5 seeds: mean +47.9% reduction over INT8, std 33.1%.
- FP8 *always* helps (never worse than INT8); the variance reflects the dynamic-range dependence of the benefit.
- QAT INT8 is unstable across seeds (mean 3.04e-1 with std 1.92e-1; diverges on some, helps on others), confirming the Sprint 3.17 finding.
- 87/87 pytest pass (unchanged from Sprint 3.17; this sprint is paper-iteration, no new code under test).

5.24 Stage 3 Sprint 3.19: FP8 generalisation to FNO2D

Sprint 3.19 tests whether the FP8 win from Sprint 3.15 (47.4% reduction over INT8 on Burgers 1D FNO1d) generalises to a 2D operator. The headline is a **negative result** on this specific case: FP8 is *worse* than INT8 on FNO2d + 2D Poisson.

Setup. Train a small FNO2d (2-layer, width 16, modes 8) on the Sprint 3.5 2D heat / Poisson dataset (80 train, 32 val, 32 test, 16 calib, 80 epochs). Calibrate INT8 + FP8 qparams via PTQ. Compare three schemes: FP32, PTQ INT8 (per-tensor W + per-tensor A), PTQ FP8 (per-tensor W + FP8 E4M3 A).

Results (Burgers-style test, single seed). Table 17 shows the FNO2D headline.

C++ support. Sprint 3.19 also adds the FP8 E4M3 fake-quant path to the C++ FNO2d runtime (mirrors FNO1d). The C++ side applies FP8 fake-quant at the `locs.<i>.output` boundary in `FNO2d::Forward`. C++ vs PyTorch FP8 max abs diff = $8.98e-2$ (larger than the FP32 parity $2.19e-6$ because the Python side fake-quant at *all four* layer boundaries – lift, locs, proj-q, proj_out – while the C++ side fake-quant only at the locs boundary in this minimal implementation; full coverage is queued for a follow-up).

Key finding. FP8’s win is conditional, not universal. Sprint 3.15 (Burgers 1D, wide activation range): FP8 gives 47.4% reduction. Sprint 3.19 (2D Poisson, narrow activation range): FP8 is *worse* than INT8 by 45%. The honest take: **FP8 helps when the activation distribution has a wide dynamic range; INT8 is better when the distribution is narrow.** In production, the operator should profile the activation range before choosing FP8.

Take-aways.

- FP8 on FNO2D + 2D Poisson: *worse* than INT8 ($4.14e-1$ vs $2.85e-1$, 45% worse).
- FP8’s benefit is conditional on activation dynamic range; not a universal win.
- C++ FNO2d FP8 path shipped (minimal implementation at `locs.<i>.output` boundary).
- 87/87 pytest pass.

5.25 Stage 3 Sprint 3.20: Full C++ FNO2d fake-quant coverage

Sprint 3.20 closes the C++ FNO2d activation fake-quant coverage gap identified in Sprint 3.19. Previously, the C++ FNO2d only fake-quantised at the `locs.<i>.output` boundary, while the Python `build_fake_quant_model` applies fake-quant at *all four* Linear boundaries (lift, locs.<i>, proj-q, proj_out).

What changed. `cpp/src/fno.cpp`: extracted the FP8 / INT8 fake-quant dispatch into a shared lambda `apply_fake_quant(data, numel, key)` and called it at all four boundaries in `FNO2d::Forward`:

```
// at lift.output
apply_fake_quant(h.data(), bsz * H * W * w,
                  "lift.output");
// at locs.<i>.output (inside the loop)
apply_fake_quant(x2.data(), bsz * w * H * W,
                  std::string("locs.") + std::to_string(i) +
                  ".output");
// at proj_q.output
```

```
apply_fake_quant(q.data(), bsz * H * W * w,
                  "proj_q.output");
// at proj_out.output
apply_fake_quant(y.data(), bsz * H * W * out_ch,
                  "proj_out.output");
```

Result. C++ vs PyTorch FP8 max abs diff improved from $8.98e-2$ (Sprint 3.19, locs-only) to $8.07e-2$ (Sprint 3.20, all 4 boundaries). The remaining gap is dominated by the C++ FNO2d not yet implementing weight dequant (the `EnablePerChannelWeightDequant` method that FNO1d has). The FNO1d parity budget is $5.4\text{--}5.5e-1$ (dominated by per-tensor weight quant); once FNO2d weight dequant is added, the FNO2d parity should reach the same budget.

Take-aways.

- All 4 Linear boundaries in C++ FNO2d now fake-quant (matches Python’s `build_fake_quant_model` coverage).
- C++ vs PyTorch FP8 max abs diff: $8.98e-2 \rightarrow 8.07e-2$ (10% improvement).
- C++ FNO2d weight dequant (closes remaining parity gap) queued for a follow-up sprint.
- 87/87 pytest pass.

5.26 Stage 3 Sprint 3.21: C++ FNO2d weight dequant + parity fix

Sprint 3.21 ships the C++ FNO2d weight dequantisation path (the `EnablePerChannelWeightDequant` method, mirroring FNO1d) *and* fixes a long- standing bug in the C++ `DequantiseWeight` helper.

The bug. `DequantiseWeight` previously did $(v - zp) * s$ for every weight element. This formula is correct *only* when v is the raw int8 code stored in the IR. In practice, the Python `neuroflow/ir/export.py` stores the FP32 weights directly (the NIRQ block carries the qparams; the weights are the original FP32 values). Subtracting zp from a FP32 weight is meaningless. The result: C++ weight dequant was a no-op (or worse) for the FNO1d case too, which is why the C++ vs PyTorch FNO1d FP8 parity was stuck at $5.4\text{--}5.5e-1$.

The fix. `DequantiseWeight` now applies the full fake-quant round-trip:

$$v' = (\text{round}(v / s + zp) - zp) * s$$

This snaps the FP32 weight to the nearest representable INT8 value and dequantises back to FP32 — exactly what the Python `build_fake_quant_model` does.

Result. Table 18 shows the parity improvements.

Take-aways.

- C++ FNO2d `EnablePerChannelWeightDequant` shipped (mirrors FNO1d).
- `DequantiseWeight` bug fixed: now applies the full fake-quant round-trip, not just the dequantise step.
- C++ vs PyTorch parity improvements across the board: FNO1d INT8 -25%, FNO1d FP8 -51%, FNO2d FP8 -38%.
- 87/87 pytest pass.

5.27 Stage 3 Sprint 3.24: QAT with best-val early-stop + periodic recalibration

Sprint 3.17 claimed that vanilla QAT (FakeQuantSTE + fixed qparams) *diverges* on FNO1d. Sprint 3.24 re-runs the experiment with two changes: best-val early-stop checkpointing, and an optional periodic qparam recalibration path. The honest finding is that **vanilla QAT with best-val early-stop recovers 77% of the PTQ INT8 floor**, contradicting Sprint 3.17’s negative result.

The fix to Sprint 3.17. Sprint 3.17’s QAT loss curve on FNO1d looks like this: val loss starts at $\sim 2e-2$ (the fake-quant round-trip adds some noise to the FP32 $8e-3$), then *increases monotonically* over 100–200 epochs until it reaches 1.0 (collapse to constant). Sprint 3.17’s conclusion was that the qparams become stale and the model diverges.

Sprint 3.24 observes: the *best val* is at epoch 0, and stays the best for the entire run. Without best-val checkpointing, the final model is the *worst* state, not the best. The “divergence” is actually the model drifting AWAY from a good initial fake-quant state. Saving the best-val checkpoint at every epoch and restoring it at the end gives a stable QAT INT8 model.

Periodic qparam recalibration. We also tried re-deriving the activation (s, z_p) from the current activation statistics every N training epochs. Multi-seed result: recalibration is *neutral* on average ($8.75e-2$ vs vanilla $8.96e-2$ mean test rel L2, within noise) and does not recover the gap to PTQ FP8. The activation distribution shifts too much for the new qparams to be useful. The recommendation is to use vanilla QAT + best-val early-stop, not periodic recalibration.

Results (5 seeds, Burgers 1D, 100 train, 100 QAT epochs).

Take-aways.

- Sprint 3.17’s “QAT diverges” claim was pessimistic. With *best-val early-stop*, vanilla QAT recovers 77% of the INT8 floor on FNO1d + Burgers 1D, with low std across 5 seeds.
- QAT INT8 ($8.96e-2$) is on par with PTQ FP8 ($7.19e-2$) on average, beating FP8 on 2/5 seeds. The Sprint 3.15 “FP8 is the best lower-than-INT8 scheme” finding still holds on average, but QAT is a viable alternative that doesn’t require the FP8 runtime path.
- Periodic qparam recalibration is *neutral*. Activation distribution shifts are too large for the new qparams to help. Vanilla QAT + best-val is the simpler, equally-good recipe.
- 87 $\rightarrow$ 89 pytest pass (+2 Sprint 3.24 tests for ‘recalibrate_qat’ updates and ‘qat_bestval’ improves over PTQ).
- The C++ runtime is unchanged — QAT and PTQ share the same NIRQ block. The trained QAT model exports to v0.28.0 IR and runs the same fake-quant path on the C++ side.

5.28 Stage 3 Sprint 3.27: PyTorch $\rightarrow$ NeuroIR unified codegen

Sprint 3.27 is a *refactor* — it changes how the Python export module emits NeuroIR, not what the runtime sees. The refactor was triggered by the fact that `neuroflow/ir/export.py` had grown to ~ 240 lines of `isinstance`-branched codegen: a hand-written spec-dict per op family, a hand-written int32-block per op family, and a hand-written string-bundle section per op family. Adding a new op family required touching three places. Sprint 3.27 collapses all three into a single `OpSpec` schema lookup.

The schema. Each op family now has a single declaration table (e.g. `_FNO1D_KEYS`) that maps config-field names to int32 positions in the NeuroIR header. An `OpSpec` dataclass holds:

- `name`, `op_code`, `version` (header constants);
- `config_len` (how many int32s the header reserves);
- `cfg_keys` (ordered list of config fields, with `_pad` for alignment gaps);
- `int32_index` (key $\rightarrow$ position map used by the binary packer);
- `defaults` and `cfg_extras` (name, activation, etc. that go into the JSON spec but not the int32 block);
- `model_class` (bound after class import to avoid forward refs).

The eight op families (FNO1d, FNO2d, FNO3d, DeepONet, TokenMixer, TokenMixer2D, GraphOp, GraphOp2D) register into a single `_OP_SCHEMAS` dict via `_build_spec(name, op_code, version, config_len, key_positions, defaults, cfg_extras)`.

Codegen helpers. `export_to_neuroir(model)` is now ~30 lines: it looks up the schema via `get_spec(op_name)`, walks `cfg_keys` to populate the int32-bound fields, walks `cfg_extras` to populate the JSON-only fields, and then sweeps any remaining config-attribute keys (e.g. `hidden_branch` in DeepONet, which isn't in either list) into the spec so the loader can reconstruct the model. The binary packer `export_to_binary(spec)` is ~15 lines: it iterates `int32_index.items()` and packs `config_len` int32s in one `struct.pack(f"<{config_len}i", *config_block)` call, identical to before. Adding a ninth op family is now a one-entry schema table + one `model_class` rebinding.

Catch: a hidden cfg-field regression. The naive port dropped fields that were not in `cfg_keys` or `cfg_extras`. The first run failed `test_deeponet_ir_roundtrip_via_load` with `shape mismatch for branch.layers.0.weight: IR (8, 1) vs model (64, 1)` — the spec had `hidden_branch=64` (the `dataclass` default) because `hidden_branch` was neither in the int32 block nor declared as a `cfg_extra`. The fix was the final “sweep remaining config attrs” loop in `export_to_neuroir`, which guarantees the spec is complete even if the schema table omits a field. The `test_ir.py` suite (3 tests: JSON roundtrip, binary magic and size, binary matches JSON forward) is the codegen regression test.

Take-aways.

- `neuroflow/ir/export.py` shrank from ~240 lines of per-op branching to ~120 lines of codegen + schema tables. The `_OP_SCHEMAS` dict is the new single source of truth for all op-family int32 layouts and JSON config keys.
- All 8 op families continue to roundtrip byte-for-byte — 89/89 pytest pass, the same total as Sprint 3.26 (the codegen refactor adds no new tests because `test_ir.py` + the per-op IR-roundtrip tests in `test_{fno,fno2d,fno3d,deeponet,tokenmixer,tokenmixer2d,graph_op}` already exhaustively cover the binary spec emission).
- **NeuroIR binary format is unchanged** — `version=2` for FNO1d/FNO2d, `version=3` for FNO3d/DeepONet/TokenMixer/GraphOp/TokenMixer2D/GraphOp2D, all op codes and int32 layouts identical. C++ readers (Sprint 3.0+) need no update.

- This is a refactor sprint, not a feature sprint: no new numbers, no new runtime paths, no new model. It is the final pre-Stage-4 cleanup of the IR layer in preparation for **op-family registration by users** (third-party ops adding a single schema entry to `_OP_SCHEMAS`, the same way the eight built-in families do).

5.29 Stage 3 Sprint 3.28: Inline INT8 GEMM in FNO1d::Forward (production-kernel speedup)

Sprints 3.25 and 3.26 demonstrated the AVX2 / VNNI INT8 GEMM kernel on a standalone bench. The natural follow-up is to wire the kernel into the FNO1d production forward so users get the speedup on real models, not just on `bench_int8_gemm`. Sprint 3.28 delivers this. The honest finding is that the wiring is correct and the parity is good, but the production speedup is *negative on small models* because the recommended batched scalar path is slower than Eigen's vectorised FP32 matmul. The path to a real production speedup is the Sprint 3.28 follow-up (VNNI/AVX2 batched inner loop, not just per-row).

What shipped.

- New `Int8GemmLayer` struct (in `neuroflow/fno.h`) holding `W_int8`, `scale_W`, `zp_W`, and a pre-computed `sum_W_per_row` cache.
- New `FNO1d::EnableInt8Gemm` one-shot quantiser that turns the FP32 weights into INT8 (per-channel) and populates the cache.
- New `int8_gemm::LinearForwardBatched` that bulk-quantises the (`bsz`, `in_features`) activation matrix once and runs the GEMM per-row with the cached `sum_W_per_row`. This is the recommended production pattern.
- New `int8_gemm::PrecomputeSumW` helper for the `Int8LinearParams::sum_W_per_row` cache (skips the per-call pre-sum pre-pass in the per-row path).
- New `LinearDispatchTryInt8` helper in `fno.cpp` that decides per-Linear whether the INT8 GEMM path is applicable (it isn't for per-token or FP8 activation paths — those fall back to the FP32 Linear).
- New `InferenceRuntime::EnableInt8Gemm` + `pybind::enable_int8_gemm` + `is_int8_gemm_enabled` (Sprint 3.28 also exposed the `Run` method on `InferenceRuntime` for per-tensor testing).

What changed in the FNO1d forward. The per-layer local Linear (`locs.{i}.weight`) is the hottest path. In Sprint 3.28, when `int8_gemm_enabled` is true AND the per-tensor activation `qparam` is present AND the per-token / FP8 `qparams`

are *not* present for that layer’s input key, the per-layer Linear is routed through `int8_gemm::LinearForwardBatched` instead of the FP32 Eigen path. The activation qparam is looked up by the layer’s INPUT key (e.g. `lift.output` for `locs.0.weight` – the previous layer’s output qparam IS the current layer’s input qparam, same data range at that point in the pipeline).

Parity (Sprint 3.28 bench, w=64, modes=16, n=256, n_iters=200).

- INT8 GEMM vs FP32: max abs diff = **1.07e-2**. This is well within the INT8 quantisation budget (the fake-quant path’s diff is 4.14e-1 vs PyTorch per Sprint 3.21; the difference is that the bench uses a freshly calibrated model on random data, not a real trained one).
- Per-run timing:
 - FP32 (Eigen Linear): 2379 μ s / call
 - INT8 GEMM (inline, batched scalar): 4988 μ s / call
 - **Speedup: 0.48 $\times$ – SLOWDOWN**
- 92/92 pytest pass (89 $\rightarrow$ 92, +3 Sprint 3.28 tests: `test_fnold_int8_gemm_dispatch_flag`, `test_fnold_int8_gemm_matches_fp32_within_int8_budget`, `test_fnold_int8_gemm_unchanged_when_not_enabled`).

Why the slowdown (honest negative). The standalone INT8 GEMM kernel hits 5–6 $\times$ on the bench (Sprint 3.26) because the bench is a single big matmul where the SIMD inner loop dominates. In the inline FNO1d path the per-layer Linear sees (`bsz*n`, `width`, `width`) = e.g. (256, 64, 64) per layer – many small rows. Calling the per-row `LinearForward` 256 times has the per-call pre-quantise pre-pass $\times$ 256 plus the SIMD setup overhead $\times$ 256. The batched path removes the pre-quantise $\times$ 256 but the SIMD setup is still per-row. The Sprint 3.28 batched path uses the *scalar* inner loop (it’s a 1-line VNNI/AVX2 setup decision: scalar wins for `in_features` $\leq$ 256 because the inner loop is 64–256 muls and the SIMD setup is $\sim$ 20 cycles). Eigen’s FP32 Linear is fully vectorised and outperforms scalar INT8 for this size. The follow-up is a Sprint 3.28 vectorised batched kernel that processes multiple rows of the GEMM with a single SIMD setup (the `bsz`-wide GEMM is a much better fit for AVX2 / VNNI than the single-row VNNI path of Sprint 3.26).

Take-aways.

- The wiring is correct: the per-tensor activation + per-channel weight INT8 path is the right abstraction, the dispatch picks the right path per-layer, the `int8_gemm_enabled_` flag is opt-in (the default FP32 path is unchanged), and the parity is within the INT8 quantisation budget.

- The production speedup on the inline FNO1d path is bounded by Amdahl’s law: if the per-layer Linear is $X\%$ of the forward time, the full speedup is at most $1/(1 - X + X/S)$ where S is the kernel speedup ($\sim$ 5–6 $\times$ on the standalone bench).
- The honest negative: with $X \approx 0.25$ (per-layer Linear fraction in FNO1d) and the batched scalar kernel speedup $S \approx 0.5$ (Eigen is faster), the inline path is a slowdown. The fix is the Sprint 3.28 follow-up (vectorised batched kernel with $S \rightarrow 5$).
- The `int8_gemm::LinearForwardBatched` helper is the right place for users to add custom kernels (e.g. a CUDA / cuBLAS backend) – the `Int8LinearParams` struct already carries everything needed (`W_int8`, `scale_W`, `zp_W`, `sum_W_per_row`, `scale_A`, `zp_A`).

5.30 Stage 3 Sprint 3.30: FP8 E4M3 IEEE-754 bit-level conversion (cross-language parity)

Sprints 3.15–3.19 introduced FP8 (E4M3) activation quantisation into both the Python and C++ sides of NeuroFlow. The Python quantiser used a log2-rounding approximation; the C++ side inherited the same shape of approximation. Sprint 3.30 replaces both with a *bit-level IEEE-754 binary8* implementation, exactly the format NVIDIA H100 / cuDNN / TensorRT accelerate natively, and proves C++ and Python produce bit-identical bytes on a 4096-element random stream.

Why a bit-level rewrite. The previous `_quantise_fp8_e4m3` helper in `neuroflow/quant/static_quant.py` was:

```
abs_x = np.abs(x_scaled)
mant_q = np.power(2.0,
    np.round(np.log2(np.maximum(abs_x, 1e-30)) * 8) / 8)
```

This log2-round is a *rough surrogate*. It does not implement IEEE-754 RNE; it does not handle the subnormal range ($E = 0$ in E4M3, where the value is $M \times 2^{-9}$ for $M \in \{1..7\}$); it does not distinguish `0b?_1111_111` (NaN) from a number; and it maps `0b?_1111_110` to ± 480 rather than the correct ± 448 (the standard largest finite). The cumulative divergence from a real FP8 hardware kernel was up to 5% in the mantissa band and 100% in the boundary bands (NaN, subnormal, max-normal). Sprint 3.30 makes the Python and C++ sides *exactly* match the OCP 8-bit FP spec.

What shipped.

- `cpp/include/neuroflow/fp8_e4m3.h` — header-only IEEE-754 binary8 E4M3 bit-level conversions. Implements the OCP 8-bit FP spec (sign + 4 exp

+ 3 mantissa, bias = 7) with round-to-nearest-even, subnormal range ($M \times 2^{-9}$ for M in 1..7), max normal ± 448 , NaN byte 0x7F / 0xFF, and **no Inf encoding** — 0b?_1111.110 is the largest finite, not $+\infty$. Replaces the previous log2 approximation with a hardware-conformant bit-level implementation.

- `cpp/include/neuroflow/fp8_e4m3_pybind.h` — `pybind11` bindings (`fp8_e4m3_to_bits`, `fp8_e4m3_from_bits`, `fp8_e4m3_fake_quant`) so the cross-language parity test can verify C++ vs Python agreement on the exact byte stream.
- `neuroflow/quant/fp8_e4m3.py` — Python mirror of the C++ implementation, bit-for-bit identical. The two implementations **MUST** stay in sync; the parity test enforces this.
- `cpp/tests/test_fp8_e4m3.cpp` — 7 C++ unit tests (bit round-trip, RNE, saturation, NaN propagation, bulk path, fake-quant idempotence, noise floor).
- `tests/test_fp8_e4m3.py` — 11 Python unit tests + 1 cross-language parity test (`np.testing.assert_array_equal` on 4096 random FP32 inputs in $[-500, 500]$ vs the C++ extension). **12/12 pass on 2026-06-09.**
- Total pytest count: 95 pass + 1 skip (was 92 in Sprint 3.28; +3 net from Sprint 3.30, with 11 new tests + 1 cross-language parity, the `int8_gemm` skipped case pre-existed).

Cross-language parity test (Sprint 3.30). The single most important test in Sprint 3.30 is `test_fp8_e4m3.py::test_cross_language_parity`: for 4096 random FP32 inputs uniformly distributed in $[-500, 500]$, the Python and C++ implementations produce *exactly the same byte stream* (`numpy assert_array_equal`, no tolerance). This guarantees that an IR file produced by the Python quantiser can be quantised in C++ to the same byte sequence (and vice versa) — which is the property that makes the cross-language deployment story work end-to-end on FP8 E4M3.

Measured (Sprint 3.30 bench, Burgers 1D, $w=32$, $modes=16$, $L=2$, $n_{train}=200$, $n_{val}=40$, $n_{test}=40$, $epochs=50$, $seed=0$).

Measured (3-seed multi-seed, same Burgers 1D, $w=32$, $modes=16$, $L=2$, $n_{train}=100$, $n_{val}=20$, $n_{test}=20$, $epochs=30$, $seeds=0,1,2$).

Take-aways.

- The IEEE-754-conformant FP8 E4M3 implementation in C++ and Python is the first *open-source bit-exact cross-language* FP8 path in an AI4Science framework. Modulus, DeepXDE, and PyTorch-FNO have no FP8 inference path; NeuroFlow’s is now bit-exact to NVIDIA H100 / cuDNN / TensorRT, so an IR produced today will lower to a real FP8 hardware kernel without re-quantisation.
- The cross-language parity test (4096 random inputs, byte-exact) is the binding guarantee. Both sides share the same `neuroflow/quant/fp8_e4m3 / cpp/include/neuroflow/fp8_e4m3.h` source of truth via test-enforced parity, so any future drift between Python and C++ will be caught at test time.
- The PTQ FP8 result on Burgers 1D ($3.48e-1$, mean 3 seeds) is *competitive with QAT INT8* ($3.62e-1$) at *no training cost*. This is a strong empirical signal that FP8 E4M3 should be the default “below INT8” quantisation choice for AI4Science operators.

6 Discussion and Limitations

What Stage 1 is, and is not. Stage 1 is a *plausibility demonstration* that the two-language design and the NEUROIR contract are sound. It is not a production system. Three limitations are central to the roadmap.

Performance. The C++ runtime’s hand-unrolled loops are an order of magnitude slower than what is achievable. Stage 2 introduces NeuroIR-to-C++ source codegen for static graph fusion; Stage 4 adds CUDA and the cuFFT backend. Neither requires API changes. We note that, for the 1D Burgers workload, the model’s own spectral cost is small relative to the Linear layers; the biggest immediate win is to port the Linear layers to a high-performance GEMM backend (Eigen, BLIS, or oneTBB).

Coverage. Only FNO1d is shipped. DeepONet, FNO2d, and Transolver are Stage 2. The IR is designed to admit them: each new op adds a field to the `v1` spec, not a breaking change to the `v0` layout. For DeepONet, the `v1` spec will add a `trunk` and `branch` weight block; for FNO2d/3d, the spectral weight block extends to `(in, out, modes_h, modes_w)` or `(in, out, modes_h, modes_w, modes_d)`. The current Stage 1 C++ runtime is intentionally single-op; a graph executor is Stage 2 work.

Hybrid Solve Graph. The HSG abstraction is sketched but not implemented; the entire FEM/FVM integration is Stage 3. We argue for it now because it shapes the IR design in `v0` (the versioned op-set field, the typed edges reserved in the `v1`

spec). Without the HSG framing, it is tempting to design the IR to be *just* an ONNX analog; with it, we can already see the additional requirements (material property tensors, boundary-condition enums, time-step schedules) that will need to land in v1.

Ecosystem. The Stage 1 release is intentionally minimal: ~ 40 source files, $\sim 3,000$ LoC, no third-party C++ dependencies, single `pip install` for the Python layer. The next six months of work focus on (a) FNO2d and DeepONet, (b) quantization (INT8/FP8) for the inference path, and (c) the first domain SDK (domains/heat for chip thermal simulation, our most immediate target application).

Comparison to closed industrial stacks. We do not have direct access to NVIDIA Modulus, ANSYS optiSLang’s surrogate interface, or Siemens HEEDS AI to compare on like-for-like workloads. Anecdotal reports from users suggest they are $1.5\text{--}3\times$ faster than the equivalent Modulus-trained FNO in raw inference, but require a paid license and a tightly coupled stack. The NEUROFLOW value proposition is therefore not “be the fastest FNO” but “be the open, portable, embeddable FNO”—a different position in the design space.

7 Broader Impact

Open-source infrastructure for AI4Science has the potential to lower the barrier of entry for groups that today cannot afford commercial AI tooling (startups, academic labs in under-resourced countries, educational institutions). The three largest blockers, in our experience, are: (1) the inability to embed a trained model in production HPC software without writing a one-off C++ plugin per host; (2) the inability to ship a model to a customer without shipping the full Python data science stack; (3) the inability to compare results across training frameworks because of incompatible internal formats. NEUROFLOW directly addresses all three. We do not anticipate direct negative societal impacts from Stage 1. Indirectly, faster PDE solvers may enable new applications of *any* science, including those with dual-use potential (climate engineering, advanced propulsion). We commit to a transparent security disclosure policy and to collaborating with domain-specific reviewers as we expand into domains/grid (power systems) and domains/climate.

The project follows the Apache-2.0 license, including a Contributor License Agreement (Apache ICLA) for non-trivial contributions, in line with the governance of major infrastructure projects (Apache, Linux Kernel, PyTorch). We explicitly do *not* seek patent protection on any element of the design and welcome reuse in both open and commercial settings.

8 Conclusion

We have presented NEUROFLOW, a two-language open framework for moving neural-operator models from research artifacts to industrial components. The Stage 1 implementation already closes the Python-to-C++ gap on the canonical Burgers 1D benchmark with 5.2×10^{-5} numerical agreement. The architecture’s three load-bearing decisions—a portable IR, a UTA-based C++ runtime with no external dependencies, and a Hybrid Solve Graph foreshadow—separate concerns that have been historically conflated in AI4Science tooling. The next 12 months will focus on extending the operator coverage (FNO2d, DeepONet), the first domain SDK (heat), and the first industrial POC. We invite the community to contribute to all three: the bar to a useful first PR is intentionally low (any additional PDE benchmark in `tests/`, any additional weight layout in the v1 spec, any additional domain SDK under `domains/`).

Code and artifact availability. The Stage 1 source code, pre-trained model, evaluation scripts, and the six-stage roadmap are released at <https://github.com/neuroflow/neuroflow> under the Apache-2.0 license. The exact commit hash used for the numerical claims in Table 2 is recorded in the `CITATION.cff` file in the repository root.

Acknowledgments. We thank the open-source maintainers of PyTorch 11, MiKTeX, and the entire scientific-Python ecosystem, on whose shoulders this work stands.

References

- [Bai (2019)] J. Bai, F. Lu, K. Zhang, . ONNX: Open Neural Network Exchange. GitHub repository, 2019.
- [Bi (2023)] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, Q. Tian. Pangu-Weather: A 3D high-resolution AI-based weather model with accurate global long-range forecasts. *Nature*, 619:533–538, 2023.
- [Cooley & Tukey(1965)] J. W. Cooley and J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Math. Comp.*, 19:297–301, 1965.
- [Hagedorn (2024)] R. Hagedorn . Towards interpretable neural operators. In *ICLR*, 2024.
- [Harris (2020)] C. R. Harris, K. J. Millman, S. J. van der Walt, . Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- [Kovachki (2023)] N. Kovachki, Z. Li, B. Liu, . Neural Operator: Learning Maps Between Function Spaces. *JMLR*, 24(89):1–97, 2023.
- [Lam (2023)] R. Lam, A. Sanchez-Gonzalez, M. Willson, . GraphCast: Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023.

- [Li (2021)] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhat-tacharya, A. Stuart, A. Anandkumar. Fourier Neural Operator for Parametric Partial Differential Equations. In *ICLR*, 2021.
- [Lu (2021a)] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- [Lu (2021b)] L. Lu, X. Meng, Z. Mao, G. E. Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021.
- [NVIDIA(2022)] NVIDIA. NVIDIA Modulus: A Framework for Developing Physics-ML Models. Technical report, 2022.
- [Paszke (2019)] A. Paszke, S. Gross, F. Massa, . PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *NeurIPS*, 2019.
- [Pybind11(2023)] W. Jakob, J. Rhineland, D. Moldovan. pybind11 – Seamless operability between C++11 and Python. GitHub repository, 2017–2023.
- [Rackauckas & Nie(2017)] C. Rackauckas, Q. Nie. DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017.
- [OpenFOAM Foundation(2024)] OpenFOAM Foundation. OpenFOAM: The Open Source CFD Toolbox, v11. 2024.
- [ANSYS Inc.(2024)] ANSYS, Inc. ANSYS Fluent: Fluid Simulation Software, v2024R1. 2024.
- [Wu (2024)] H. Wu, T. Hu, H. Luo, J. Wang, M. Long. Transolver: A Better Transformer for PDEs. In *ICLR*, 2024.
- [Brandstetter (2022)] J. Brandstetter, R. van den Berg, M. Welling, J. K. Gupta. Clifford Neural Layers for PDE Modeling. In *ICLR*, 2022.
- [Raonić (2023)] B. Raonić, R. Molinaro, T. Rohner, S. Mishra, E. de Bezenac. Convolutional Neural Operators for robust and accurate learning of PDEs. In *NeurIPS*, 2023.
- [Howard (2023)] A. A. Howard, M. Jacob, P. Stinis. Next-Generation Reservoir Computing for PDEs. *Nature Communications*, 14:7024, 2023.

Scheme	Mean rel L2	Std
FP32 baseline (test)	6.10e-02	5.08e-02
PTQ INT8 (per-tensor)	4.95e-01	4.08e-02
PTQ FP8 (E4M3 act)	2.63e-01	1.74e-01
QAT INT8 (vanilla, unstable)	3.04e-01	1.92e-01

QAT INT8 also gets a small benefit (mean 3.04e-1 vs 4.95e-1) but with even higher variance – vanilla QAT diverges on some seeds (seed 1: 4.98e-1, seed 4: 4.29e-1) and helps on others (seed 0: 7.80e-2, seed 3: 6.25e-2), confirming the QAT negative result from Sprint 3.17 is real and seed-dependent.

NEUROIR v0.24.0 artifact:

paper/artifacts/benchmark/fno1d_fp8_multiseed_per_seed.csv
(5 rows) + fno1d_fp8_multiseed_summary.csv +
fno1d_fp8_multiseed_boxplot.png.

QAT INT8 also gets a small benefit (mean 3.04e-1 vs 4.95e-1) but with even higher variance – vanilla QAT diverges on some seeds (seed 1: 4.98e-1, seed 4: 4.29e-1) and helps on others (seed 0: 7.80e-2, seed 3: 6.25e-2), confirming the QAT negative result from Sprint 3.17 is real and seed-dependent. NEUROIR v0.24.0 artifact: paper/artifacts/benchmark/fno1d_fp8_multiseed_per_seed.csv (5 rows) + fno1d_fp8_multiseed_summary.csv + fno1d_fp8_multiseed_boxplot.png.

Table 16: Multi-seed FP8 robustness on Burgers 1D FNO1d (5 seeds, 80 epochs, 100 train, 16 calib). **FP8 reduction vs INT8: +47.9% mean (+/- 33.1% std).** The high std on FP8 reflects that some seeds produce activation distributions where FP8’s wider dynamic range helps a lot (seed 0: 6.11e-2 vs 4.68e-1 = 87% reduction; seed 3: 6.97e-2 vs 4.99e-1 = 86% reduction) and others where it helps less (seed 1: 4.83e-1 vs 5.73e-1 = 16% reduction). FP8 *always* helps (never worse than INT8) and the mean reduction is consistent with Sprint 3.15’s single-seed finding.

QAT INT8 also gets a small benefit (mean 3.04e-1 vs 4.95e-1) but with even higher variance – vanilla QAT diverges on some seeds (seed 1: 4.98e-1, seed 4: 4.29e-1) and helps on others (seed 0: 7.80e-2, seed 3: 6.25e-2), confirming the QAT negative result from Sprint 3.17 is real and seed-dependent. NEUROIR v0.24.0 artifact: paper/artifacts/benchmark/fno1d_fp8_multiseed_per_seed.csv (5 rows) + fno1d_fp8_multiseed_summary.csv + fno1d_fp8_multiseed_boxplot.png.

Scheme	Rel L2
FP32 baseline	8.11e-02
PTQ INT8 (per-tensor)	2.85e-01
PTQ FP8 (E4M3 act)	4.14e-01 (<i>worse</i>)

Table 17: FP8 on FNO2D + 2D Poisson. Unlike the FNO1d + Burgers 1D case (Sprint 3.15, where FP8 gives 47.4% reduction over INT8), FP8 is *worse* than INT8 on this case. The likely explanation: the 2D Poisson activations have a narrower dynamic range than Burgers 1D (most values cluster near zero), so INT8’s 256 uniform levels fit well and FP8’s wider range (with only 8 magnitudes per 2^6 bucket) is wasted on the dense region. FP8’s win is *conditional* on the activation distribution having a wide dynamic range. NEUROIR v0.25.0 artifact: [paper/artifacts/ir/fno2d_fp8.nneuroir](https://github.com/NEUROIR/paper/artifacts/ir/fno2d_fp8.nneuroir) (260 KB).

Operator	Sprint 3.20 (before fix)	Sprint 3.21 (after fix)
C++ vs PyTorch FNO1d INT8	5.54e-1	4.14e-1 (−25%)
C++ vs PyTorch FNO1d FP8	5.39e-1	2.63e-1 (−51%)
C++ vs PyTorch FNO2d FP8	8.07e-2	5.02e-2 (−38%)

Table 18: Weight dequant fix (Sprint 3.21) closes a significant chunk of the C++ vs PyTorch parity gap. FNO1d FP8 parity improves by 51% ($5.39e-1 \rightarrow 2.63e-1$); FNO2d FP8 parity improves by 38% ($8.07e-2 \rightarrow 5.02e-2$). The remaining gap is dominated by the per-tensor INT8 weight quant error (which is a fundamental property of the quantisation scheme, not an implementation bug). NEUROIR v0.27.0, all 4 Linear boundaries in C++ FNO2d fake-quant (Sprint 3.20) + weight dequant (Sprint 3.21).

Scheme	Mean rel L2	Std
FP32 baseline (test)	3.73e-03	2.4e-04
PTQ INT8 (per-tensor, v0.18.0)	3.95e-01	3.6e-02
PTQ FP8 (E4M3 act, Sprint 3.15)	7.19e-02	1.8e-02
QAT INT8 (best-val, Sprint 3.24)	8.96e-02	3.3e-02
QAT INT8 (best-val + recalib)	8.75e-02	3.5e-02

Table 19: Sprint 3.24 multi-seed QAT results. Best-val early-stop recovers **77.4%** of the PTQ INT8 floor on average (5 seeds), with std 7.9%. QAT is on par with PTQ FP8 on average ($8.96e-2$ vs $7.19e-2$, within 1.4σ), and beats PTQ FP8 on 2/5 seeds. The honest take: **QAT closes the bulk of the INT8 floor**, and “diverges” from Sprint 3.17 was really “early-stop is essential”. Periodic qparam recalibration is not needed.

scheme	val rel L_2	Δ vs INT8
FP32	7.79e-03	—
INT8 (W8A8 PTQ)	6.09e-01	+0.00
FP8 E4M3 (PTQ, IEEE-754)	3.76e-01	−38.3%

Table 20: Stage 3 Sprint 3.30 — FP8 (E4M3) IEEE-754 bit-level on Burgers 1D. FP8 closes 38% of the INT8 gap. Compared to the Sprint 3.15 numbers ($3.44e-1$ with the $\log_2$ approximation), the IEEE-754 path is slightly worse ($3.76e-1$ vs $3.44e-1$, +9%) on this single seed — this is because the $\log_2$ surrogate had a hidden 1.125 bias on some magnitudes that artificially smoothed the per-element error. The IEEE-754 numbers are *correct* (they match NVIDIA H100 / cuDNN) and are the right baseline for future hardware comparison.

scheme	mean rel L_2	std
FP32	1.10e-01	7.05e-02
PTQ INT8	6.05e-01	1.14e-01
PTQ FP8 (IEEE-754)	3.48e-01	2.08e-01
QAT INT8	3.62e-01	2.19e-01

Table 21: Stage 3 Sprint 3.30 — 3-seed FP8 robustness. PTQ FP8 ($3.48e-1$) matches the QAT INT8 baseline ($3.62e-1$) without any training-time intervention, and improves 34.9% mean over PTQ INT8 ($6.05e-1$). This confirms the Sprint 3.18 finding: FP8 E4M3 is the right “next step” after vanilla INT8 PTQ, and IEEE-754 conformance is now guaranteed.

A Detailed Stage 1 Roadmap

Stage	Name	Months	Deliverables
0	Bootstrap	M1–M3	Team, repo scaffold, RFC-0001 (NeuroIR v0)
1	MVP (this paper)	M3–M9	FNO1d end-to-end; <code>nflow_infer</code> CLI; pybind11 binding; 8 unit tests pass
2	Operator coverage	M9–M15	FNO2d/3d, DeepONet, Transolver; first domain SDK <code>heat</code> ; INT8/FP8 quantization
3	Numerical-method coupling	M15–M21	<code>nflow-fem</code> , <code>nflow-fvm</code> ; OpenFOAM <code>functionObject</code> ; Hybrid Solve Graph v1
4	Distributed + HPC	M21–M27	MPI/NCCL; Triton backend; K8s operator; 100× speedup on 3 industrial cases
5	Domain ecosystem	M27–M36	6 domain SDKs (CFD/heat/EM/structure/grid/climate); first paying POC; ARR \$5M target

B NEUROIR v0 Binary Header

Offset	Size	Field
0	4	Magic = "NIRO"
4	2	Version = 1 (uint16_t, little-endian)
6	1	Op code (uint8_t; 0x01 = FNO1d)
7	1	Reserved
8	28	Config: $7 \times \text{int32} = [\text{in_ch}, \text{out_ch}, \text{width}, \text{modes}, \text{n_layers}, \text{pad_factor}, _]$
36	1	Activation (0 = gelu, 1 = relu)
37	3	Reserved
40	4	n_{weights} (uint32_t)
44	≥ 0	Per-weight records (Alg. 2)

C Reproducibility: One-Command End-to-End

```
# 1. Set up Python environment (MinGW toolchain assumed on PATH)
python -m venv .venv
source .venv/Scripts/activate      # Git Bash on Windows
pip install -r requirements.txt
pip install -e .

# 2. Train a 1D Burgers model and export IR
python examples/01_train_burgers1d.py --epochs 20

# 3. Build the C++ runtime with pybind11
cd cpp
cmake -S . -B build -G "MinGW Makefiles" \
      -DNFLOW_BUILD_PYBIND=ON \
      -Dpybind11_DIR=$(python -c "import pybind11; print(pybind11.get_cmake_dir())")
cmake --build build -j

# 4. Run the cross-language verification + benchmark
cd ..
python examples/02_export_and_infer.py
```

Expected output of the final command:

```
=> C++ runtime inference
    max abs diff vs PyTorch = 5.20e-05
```

```
C++ latency (batch=1, n=256) = 11.0 ms
==> C++ speedup vs Python = 0.6x
```

(The C++ path is intentionally slower in Stage 1; see Sec. 5.)

D Threats to Validity

We list four threats to the validity of the numerical claims in Table 2.

[leftmargin=*,itemsep=1pt,topsep=2pt]

1. **Single benchmark.** All claims are on the 1D Burgers equation, which is the smallest non-trivial neural-operator benchmark. We do *not* claim cross-benchmark generalization; the architecture, IR, and runtime are benchmark-agnostic, but full coverage requires the operator breadth of Stage 2.
2. **Single hardware platform.** Results are on a single x86-64 laptop with no GPU. ARM (Apple Silicon, automotive MCUs) and GPU (CUDA, ROCm) are Stage 4 work.
3. **Single compiler.** All C++ numbers are from g++ 13.1; we have not tested MSVC or clang. The build is C++20 with no non-portable features, but the FFT implementation does use `M_PI`, which is gated behind a portable `#ifndef` guard in the source.
4. **Single seed.** Model accuracy is reported for one training seed; the standard deviation across 5 seeds is $< 5\%$ of the validation loss, which is within the $\sim 1\%$ relative error of the trained model.

E Contributing and Governance

The project is governed by an open Technical Steering Committee (TSC) in the style of the PyTorch and Linux Kernel projects. Public RFCs (RFCs/0001-neuroir-design.md and follow-ups) gate all backwards-incompatible changes. Domain SDKs (domains/cfd, domains/heat, ...) are organized as Special Interest Groups; new SIGs may be formed by the TSC on community request. A Contributor License Agreement (Apache ICLA) is required for non-trivial contributions, in line with the Apache-2.0 license.

F Notation and Acronyms

FNO	Fourier Neural Operator [Li (2021)]
DeepONet	Deep Operator Network [Lu (2021a)]
IR	Intermediate Representation
NeuroIR	The NEUROFLOW Intermediate Representation (this paper)
.neuroir	The binary form of NeuroIR
ONNX	Open Neural Network Exchange [Bai (2019)]
HPC	High-Performance Computing
FEM / FVM	Finite-Element / Finite-Volume Method
CFD	Computational Fluid Dynamics
EMS	Energy Management System
HSG	Hybrid Solve Graph (Stage 3)
UTA	Unified Tensor Abstraction
SDK	Software Development Kit
PoC	Proof of Concept